



INTRODUCCIÓN A LLM

SOFTWARE AVANZADO 2026

Marco Tulio Aldana Prillwitz

CONTENIDO

01

INTRODUCCIÓN A LLM

Introducción a Large
Language Models

02

HISTORIA DE LLM

Historia de LLM

03

PANORAMA A LLM

Panorama y Definición de LLM

04

OBJETIVO DE LLM

Objetivo y cómo funciona
LLM

05

VENTAJAS DE LLM

Enumerar las ventajas de LLM

06

DESVENTAJAS DE LLM

Enumerar las desventajas de
LLM

LLM: software que trabaja con lenguaje

MODELOS DE LENGUAJE GRANDES

Los modelos de lenguaje grandes convierten texto y código en una interfaz programable.



Historia · funcionamiento · ingeniería de software · adopción responsable

Un LLM predice el siguiente token

Su aparente comprensión emerge al aprender patrones estadísticos sobre enormes corpus.

- 01 **Entrada** → **tokens** → **vectores**
- 02 Transformer relaciona el contexto
- 03 Salida → distribución de probabilidades

$$P(t_{\square} \mid t_1 \dots t_{\square-1})$$

Idea clave

Recorra el flujo de izquierda a derecha.

La escala cambió el paradigma

La evolución fue de reglas manuales a modelos fundacionales reutilizables.

- 01 **1950–2012: reglas, estadística y redes**
- 02 **2017: arquitectura Transformer**
- 03 **2018–hoy: preentrenamiento, instrucciones y multimodalidad**

1950 → 2017 →
2026

Idea clave

Presente la historia como cambios de paradigma, no como una lista de marcas.

El objetivo es modelar lenguaje reutilizable

Un modelo general aprende representaciones que luego se adaptan a múltiples tareas.

- 01 **Predecir y completar secuencias**
- 02 Seguir instrucciones y usar contexto
- 03 Transferir conocimiento con pocos ejemplos

**preentrenar →
adaptar → aplicar**

Idea clave

Diferencie el objetivo matemático de entrenamiento del objetivo de producto.

Transformer es el motor central

La atención permite ponderar relaciones entre tokens, incluso cuando están separados.

- 01 Tokenización y embeddings
- 02 Atención multicabeza + capas feed-forward
- 03 Decodificación controlada por temperatura y límites

tokens ↔ atención
→ respuesta

Idea clave

Explique la atención como un mecanismo de relevancia contextual.

La ganancia principal es velocidad cognitiva

Los LLM reducen el costo de pasar de intención a borrador, código o explicación.

- 01 **Automatización flexible en lenguaje natural**
- 02 **Productividad y aprendizaje asistido**
- 03 **Reutilización transversal: texto, código y datos**

**idea × contexto =
aceleración**

Idea clave

Conecte cada ventaja con un flujo real: idear pruebas, explicar código heredado, redactar documentación o transformar formatos.

Fluidez no equivale a confiabilidad

La respuesta puede sonar segura y aun ser falsa, sesgada, insegura o costosa.

01 **Alucinaciones y conocimiento incompleto**

02 Privacidad, sesgo, propiedad intelectual

03 Latencia, costo y ataques de prompt injection

**verificar • proteger •
medir**

Idea clave

Use esta diapositiva para establecer límites operativos.

En software, el valor cubre todo el ciclo

El mejor patrón es copiloto con contexto, herramientas y puntos de control.

- 01 **Análisis: requisitos, diseño y exploración**
- 02 Construcción: código, pruebas y migraciones
- 03 Operación: documentación, soporte e incidentes

planear → **construir**
→ **operar**

Idea clave

Explique que el modelo aporta más valor cuando recibe repositorio, convenciones, pruebas y definición de terminado.

No existe un único “mejor” modelo

La selección depende de calidad, costo, latencia, privacidad, contexto y licencia.

- 01 Familias propietarias: GPT, Claude, Gemini
- 02 Pesos abiertos: Llama, Mistral, Qwen, DeepSeek
- 03 Especializados: código, razonamiento y multimodalidad

calidad ↔ control ↔
costo

Idea clave

Evite convertir la comparación en un ranking fijo.

Un sistema LLM es más que el modelo

La ventaja sostenible proviene de la arquitectura, los datos y la evaluación continua.

- 01 **RAG, embeddings y bases vectoriales**
- 02 Agentes, function calling y orquestación
- 03 Guardrails, observabilidad, evals y LLMOps

**datos + modelo +
herramientas + evals**

Idea clave

Cierre con una ruta práctica: elegir un caso acotado, construir un baseline, conectar conocimiento autorizado, evaluar calidad y riesgo, y escalar sólo cuando las métricas lo justifiquen..

ITSM (IT SERVICE MANAGEMENT)

- Es una disciplina basada en procesos enfocados en alinear los servicios de tecnología proporcionados con las necesidades de las empresas, poniendo énfasis en los beneficios que puede percibir el cliente final.
- ITSM propone cambiar el paradigma de gestión de TI, por una colección de componentes enfocados en servicios usando distintos marcos de trabajo con las "mejores prácticas".
- Estos marcos de trabajo pueden ser ITIL (Information Technology Infrastructure Library) o eSCM (enabled Service Capability Model)



BIBLIOGRAFÍA

- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901. <https://arxiv.org/abs/2005.14165>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 4171–4186). Association for Computational Linguistics. <https://doi.org/10.18653/v1/N19-1423>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-T., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. <https://arxiv.org/abs/2005.11401>
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., & Lowe, R. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 27730–27744. <https://arxiv.org/abs/2203.02155>
- Tabassi, E. (2023). Artificial intelligence risk management framework (AI RMF 1.0) (NIST AI 100-1). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.AI.100-1>
- UNESCO. (2021). Recomendación sobre la ética de la inteligencia artificial. <https://www.unesco.org/es/legal-affairs/recommendation-ethics-artificial-intelligence>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30. <https://arxiv.org/abs/1706.03762>



PROMPT ENGINEERING

SOFTWARE AVANZADO 2026

Msc. Marco Tulio Aldana Prillwitz

CONTENIDO

01**DEFINICIÓN**

Definición de la arquitectura empresarial

02**HISTORIA**

Breve historia de la arquitectura empresarial

03**VISIÓN**

Describir los objetivos de la arquitectura empresarial

04**COMPONENTES**

Enumerar los componentes de la arquitectura empresarial

05**MODELO DE MADUREZ**

Definir el modelo de madurez de la AE

06**ruta**

Explicar la ruta de implementación de la AE

1. Introducción al Prompt Engineering

Un prompt es una interfaz de comportamiento

1

Traduce una intención humana en una tarea ejecutable.

2

Reduce ambigüedad mediante contexto, límites y criterios.

3

Convierte respuestas aisladas en procesos repetibles.

Prompt Engineering es la disciplina de diseñar instrucciones para que un modelo de lenguaje produzca resultados útiles, seguros y consistentes.

¿Por qué importa en ingeniería de software?

La calidad del prompt afecta todo el ciclo

Velocidad

Acelera análisis, documentación, pruebas y prototipos.

Calidad

Hace explícitos los estándares y casos límite.

Control

Facilita revisar, comparar y versionar resultados.

Riesgo

Reduce alucinaciones y acciones fuera de alcance.

2. Componentes de un Prompt

Un buen prompt combina seis piezas

1 Rol

perspectiva o especialidad esperada.

2 Objetivo

resultado concreto que se desea.

3 Contexto

datos, restricciones y audiencia.

4 Entrada

material sobre el que se trabaja.

5 Formato

estructura exacta de la salida.

6 Criterios

cómo evaluar una respuesta correcta.

De intención a especificación

La precisión se construye por capas



3. Técnicas de Prompt Engineering

Elegir la técnica según la complejidad

- ✓ **Zero-shot: instrucción directa sin ejemplos.**
- ✓ **Few-shot: ejemplos que fijan patrón y estilo.**
- ✓ **Descomposición: dividir el problema en subtarear.**
- ✓ **Crítica y revisión: generar, comprobar y mejorar.**
- ✓ **Uso de herramientas: delegar cálculo, búsqueda o código.**

Zero-shot vs. Few-shot

Los ejemplos reducen la variabilidad cuando el formato importa

Zero-shot

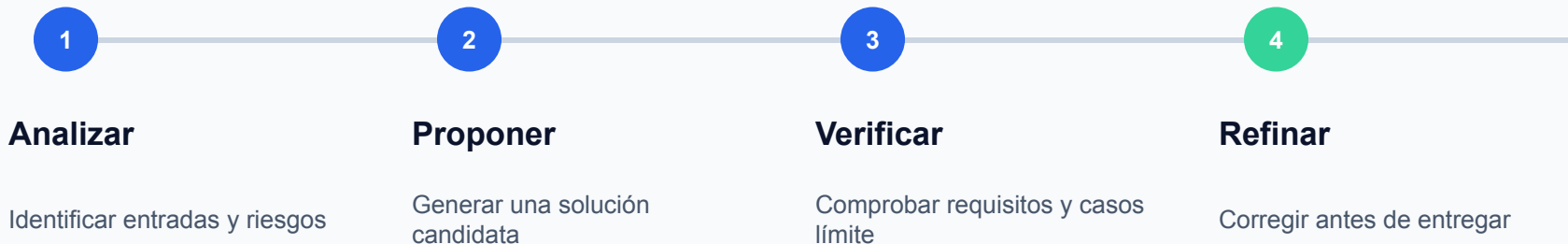
- **Rápido y económico**
- Ideal para tareas claras
- Mayor variación de estilo

Few-shot

- **Más consistente**
- Enseña casos y bordes
- Consume más contexto

Descomposición y razonamiento guiado

Separar la tarea mejora control y trazabilidad



4. Prompt Patterns

Estructuras reutilizables para problemas recurrentes

Persona + tarea

Define rol, objetivo y audiencia.

Plantilla de salida

Obliga a responder con un esquema estable.

Crítico / revisor

Evalúa una primera respuesta con una rúbrica.

Generar → verificar

Separa producción de validación.

Patrón: Contexto → Tarea → Restricciones → Salida

Una plantilla mínima para equipos de desarrollo

CONTEXTO: Servicio REST en Node.js

TAREA: Diseña pruebas para POST /orders

RESTRICCIONES: Sin mocks de red; cubrir errores 4xx

SALIDA: Tabla con caso, entrada y resultado esperado

Patrón: Generar → Criticar → Mejorar

El modelo cambia de rol para detectar fallos antes de entregar



5. Optimización de Prompts

Optimizar es un ciclo experimental, no un acto creativo aislado



Rúbrica de calidad

Evalúa el prompt con criterios observables

1

Exactitud

¿La respuesta es correcta?

2

Completitud

¿Cubre requisitos y bordes?

3

Consistencia

¿Mantiene formato entre ejecuciones?

4

Eficiencia

¿Usa contexto, costo y tiempo razonables?

6. Casos Prácticos

Un mismo marco sirve para múltiples tareas de software

- ✓ **Generación de pruebas unitarias con casos límite.**
- ✓ **Revisión de código con severidad y evidencia.**
- ✓ Documentación de API desde contratos y ejemplos.
- ✓ Diagnóstico de incidentes con hipótesis verificables.
- ✓ Migración de código bajo restricciones de compatibilidad.

Caso práctico: revisión de código

Prompt antes de producción

ROL: Revisor senior de TypeScript
OBJETIVO: Detectar defectos y riesgos de mantenimiento
CONTEXTO: API de pagos; Node 22; strict mode
CRITERIOS: Seguridad, concurrencia, errores y pruebas
SALIDA: Hallazgos P0–P3 con evidencia y corrección

Trata los prompts como código: diseña, prueba, mide, versiona y revisa.

BIBLIOGRAFÍA

1. Schulhoff, S., et al. (2024). *The Prompt Report: A Systematic Survey of Prompting Techniques*.
Revisión extensa con una taxonomía de más de 50 técnicas. Es probablemente el mejor punto de partida. [Artículo](#)
2. Sahoo, P., Singh, A. K., Saha, S., Jain, V., Mondal, S., & Chadha, A. (2024). *A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications*.
Explica técnicas, aplicaciones, modelos utilizados y limitaciones. [Artículo](#)
3. Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., & Neubig, G. (2023). *Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing*. *ACM Computing Surveys*, 55(9), 1–35.
Una referencia fundamental para comprender cómo surgió el paradigma de prompting. [Artículo](#)
4. Brown, T. B., et al. (2020). *Language Models are Few-Shot Learners*.
Trabajo de GPT-3 que popularizó el aprendizaje *zero-shot*, *one-shot* y *few-shot*. [Artículo](#)
5. Wei, J., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*.
Introduce demostraciones con pasos intermedios para tareas de razonamiento. [Artículo](#)
6. Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., & Iwasawa, Y. (2022). *Large Language Models are Zero-Shot Reasoners*.
Estudia el razonamiento paso a paso sin proporcionar ejemplos previos. [Artículo](#)
7. Wang, X., et al. (2023). *Self-Consistency Improves Chain of Thought Reasoning in Language Models*.
Propone generar varias rutas de razonamiento y seleccionar la respuesta más consistente. [Artículo](#)
8. Yao, S., et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*.
Combina razonamiento con acciones y uso de herramientas; es fundamental para estudiar agentes. [Artículo](#)
9. Yao, S., et al. (2023). *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*.
Extiende *chain-of-thought* mediante exploración, evaluación y retroceso entre varias soluciones. [Artículo](#)
10. Zhou, Y., et al. (2023). *Large Language Models Are Human-Level Prompt Engineers*.
Introduce APE, una propuesta para generar y seleccionar instrucciones automáticamente. [Artículo](#)



CONTEXT ENGINEERING

SOFTWARE AVANZADO 2026

Marco Tulio Aldana Prillwitz

CONTENIDO

01

INTRODUCCIÓN A CONTEXT ENGINEERING

Conocimiento general a IA,
LLM y Context Engineering

02

HISTORIA

Pequeña historia de Context

03

COMPRENDER

Distinguir contexto, prompt,
RAG y fine-tuning.

04

DISEÑAR

Seleccionar evidencia
relevante para cada paso.

05

EVALUAR

Comparar calidad, latencia y
consumo con un modelo fijo.

06

VERIFICAR

Crear un asistente de
documentación con fuentes y
JSON.

INTRODUCCIÓN

¿Qué es Context Engineering?

Es el proceso de **seleccionar, organizar y actualizar la información que recibe una IA** para realizar una tarea. Incluye instrucciones, datos relevantes, historial de conversación y resultados de herramientas, con el fin de generar respuestas más coherentes y confiables (Rajasekaran et al., 2025).

Referencia: Rajasekaran, P., Dixon, E., Ryan, C., & Hadfield, J. (2025, 29 de septiembre). *Effective context engineering for AI agents*.

HISTORIA

El **context engineering** tiene antecedentes anteriores a la popularización de su nombre. Su evolución puede explicarse en tres momentos:

- **2020: recuperación de información externa.** Lewis y sus colaboradores presentaron RAG, una técnica que combina modelos de lenguaje con documentos recuperados de una fuente externa. (Lewis et al., 2020).
- **2022–2023: modelos que interactúan con herramientas.** ReAct, presentado en 2022 y publicado en ICLR en 2023, propuso combinar razonamiento y acciones para consultar fuentes externas. (Yao et al., 2023).
- **2025: mayor difusión del término.** Publicaciones de Cognition, LangChain y Anthropic ayudaron a difundir el concepto de *context engineering* en el desarrollo de agentes. El enfoque pasó a abarcar la gestión conjunta de instrucciones, memoria, documentos y resultados de herramientas, además de la redacción de prompts. (LangChain, 2025; Anthropic, 2025).

HERRAMIENTAS

Para aplicar **context engineering** se utilizan herramientas que ayudan a recuperar información, administrar la memoria y evaluar las respuestas de la IA. Algunos ejemplos son:

Herramienta	¿Para qué se utiliza?
 LangChain	Organiza la información y las herramientas que recibe el modelo en cada etapa de una tarea.
 LlamaIndex	Conecta documentos y fuentes de datos con la IA, recuperando información relevante para responder consultas.
 Pinecone	Almacena representaciones numéricas de información y permite buscar contenido por similitud semántica.
 LangGraph	Administra el estado, la memoria y el flujo de trabajo de agentes que realizan tareas en varios pasos.
 LangSmith	Permite inspeccionar qué contexto utilizó un agente y evaluar la calidad de sus resultados.

CONCEPTOS : PROMPT ENGINEERING

El *prompt engineering*, o ingeniería de instrucciones, consiste en diseñar y organizar las indicaciones que recibe una inteligencia artificial para realizar una tarea. Su objetivo es comunicar claramente qué se necesita y orientar al modelo hacia respuestas útiles y pertinentes.

Un buen prompt define la tarea, aporta contexto e indica el formato esperado; también puede incluir ejemplos y restricciones. En ingeniería de software, estas instrucciones permiten solicitar explicaciones de código, propuestas de pruebas o revisiones de funciones (Rajasekaran et al., 2025).

CONCEPTOS : CONTEXT ENGINEERING

El *context engineering*, o ingeniería de contexto, consiste en seleccionar, organizar y actualizar la información que recibe una inteligencia artificial para realizar una tarea. Incluye las instrucciones, los documentos relevantes, el historial de conversación y los resultados de herramientas externas.

Su objetivo es proporcionar información suficiente y pertinente en cada etapa del trabajo. En ingeniería de software, permite incorporar código, requisitos y pruebas para orientar al modelo, considerando que su capacidad para manejar contexto es limitada (Rajasekaran et al., 2025).

CONCEPTOS : RAG

RAG (*Retrieval-Augmented Generation*), o generación aumentada por recuperación, es una técnica que combina la búsqueda de información externa con la generación de respuestas mediante inteligencia artificial. Permite que un modelo consulte documentos relevantes para responder una pregunta (Lewis et al., 2020).

Su funcionamiento consiste en recuperar fragmentos relacionados con una consulta e incorporarlos al contexto del modelo. En ingeniería de software, puede utilizarse para crear asistentes que respondan preguntas sobre documentación técnica, manuales o procedimientos internos.

CONCEPTOS : FINE TUNING

El *fine-tuning*, o ajuste fino, consiste en entrenar adicionalmente un modelo de inteligencia artificial previamente entrenado para adaptarlo a tareas o dominios específicos. Este proceso modifica parámetros del modelo, o parámetros adicionales de adaptación, utilizando ejemplos del comportamiento esperado (Hu et al., 2021).

El *fine-tuning* y el *context engineering* son prácticas complementarias: el primero adapta el modelo mediante entrenamiento, mientras que el segundo organiza la información que recibe al ejecutar una tarea. Así, un modelo especializado puede utilizar documentos, instrucciones y datos actualizados proporcionados mediante ingeniería de contexto (Rajasekaran et al., 2025).

COMPONENTES DEL CONTEXTO

Los principales componentes del *context engineering* incluyen las **instrucciones**, que definen la tarea y sus restricciones; los **datos relevantes**, que aportan información para resolverla; y los **ejemplos**, que muestran el comportamiento o formato esperado. Su selección permite orientar las respuestas del modelo hacia los objetivos de la aplicación (Rajasekaran et al., 2025).

También comprende el **historial de conversación**, la **memoria** y las **herramientas**. El historial conserva las interacciones recientes; la memoria permite recuperar información guardada; y las herramientas facilitan consultar fuentes externas o ejecutar acciones. Estos componentes deben gestionarse para proporcionar información pertinente en cada etapa del trabajo (The LangChain Team, 2025).

CONSTRUCCIÓN DEL CONTEXTO

La construcción de un sistema con *context engineering* comienza por definir la tarea, el resultado esperado y la información necesaria para realizarla. Después, se preparan instrucciones claras, se seleccionan fuentes relevantes y se establecen las herramientas que el modelo podrá utilizar (Rajasekaran et al., 2025).

Durante la ejecución, el sistema recupera datos, incorpora resultados de herramientas y conserva las decisiones importantes. También resume o elimina contenido redundante y evalúa si el contexto proporcionado mejora las respuestas. Por ello, su construcción es un proceso continuo de diseño, prueba y ajuste (The LangChain Team, 2025).

VENTAJAS

Una ventaja del *context engineering* es que **favorece respuestas más pertinentes y coherentes** al proporcionar instrucciones claras e información relacionada con la tarea. Esto ayuda al modelo a comprender los requisitos y fundamentar sus respuestas en los datos disponibles (Rajasekaran et al., 2025).

También permite **aprovechar mejor el contexto y mantener la continuidad del trabajo**. Seleccionar información, resumir contenido redundante y recuperar decisiones guardadas puede reducir el procesamiento innecesario y facilitar tareas que requieren varias interacciones (The LangChain Team, 2025).

DESVENTAJAS

Una desventaja del *context engineering* es la **complejidad de implementación y mantenimiento**. Seleccionar documentos, administrar la memoria y conectar herramientas requiere trabajo de desarrollo y evaluación. Además, recuperar información durante la ejecución puede aumentar el tiempo de respuesta (Rajasekaran et al., 2025).

Otra limitación es la **dependencia de la calidad del contexto**. Los datos incorrectos, contradictorios o irrelevantes pueden perjudicar las respuestas. Asimismo, acumular demasiada información puede aumentar el consumo de tokens, mientras que resumirla excesivamente puede eliminar detalles importantes (The LangChain Team, 2025; Rajasekaran et al., 2025).

PRESUPUESTO Y MEMORIA

- Capacidad limitada
 - Reservar espacio para la salida y un margen operativo.
- Selección de evidencia
 - Priorizar información aplicable al usuario y a la tarea.
- Continuidad
 - Resumir avances sin perder restricciones ni referencias.
- Ejemplo de 8.000 tokens
 - 4.500 evidencia, 2.000 instrucciones e historial, 1.000 herramientas, 500 margen.

EJEMPLOS PARA SOFTWARE

La información necesaria depende de la tarea y de las reglas del sistema.



Generación de SQL

Esquema, PostgreSQL, pedidos pagados y mes en UTC permiten definir la consulta.



Diagnóstico de pipelines

Logs de staging y esquema actual revelan una columna inexistente.



Implementación de APIs

Contrato, validaciones, código de referencia y pruebas guían el endpoint.

HERRAMIENTAS POR FUNCIÓN

Arquitectura inicial

Python, JSON y SQL bastan para un prototipo pequeño. Agregar componentes cuando la tarea los justifique.

Recuperación

LlamaIndex organiza ingesta y consultas sobre fuentes.

Búsqueda

PostgreSQL con pgvector permite similitud vectorial.

Orquestación

LangGraph organiza flujos con estado. MCP conecta herramientas.

Evaluación

Promptfoo permite probar variantes de aplicaciones y prompts.

CASO DE ÉXITO

Un caso de éxito es **Research de Anthropic**, que utiliza agentes para investigar temas complejos. Aplica ingeniería de contexto mediante búsquedas dirigidas, memoria externa y resúmenes de hallazgos (Hadfield et al., 2025).

En una evaluación interna, su sistema multiagente superó en desempeño al agente individual Claude Opus 4 en **90,2 %**. Este resultado corresponde al sistema completo; no demuestra el efecto aislado del *context engineering* (Hadfield et al., 2025).

BIBLIOGRAFÍA

- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901. <https://arxiv.org/abs/2005.14165>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 4171–4186). Association for Computational Linguistics. <https://doi.org/10.18653/v1/N19-1423>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-T., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. <https://arxiv.org/abs/2005.11401>
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., & Lowe, R. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 27730–27744. <https://arxiv.org/abs/2203.02155>
- Tabassi, E. (2023). Artificial intelligence risk management framework (AI RMF 1.0) (NIST AI 100-1). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.AI.100-1>
- UNESCO. (2021). Recomendación sobre la ética de la inteligencia artificial. <https://www.unesco.org/es/legal-affairs/recommendation-ethics-artificial-intelligence>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30. <https://arxiv.org/abs/1706.03762>



HARNESS ENGINEERING

SOFTWARE AVANZADO 2026

Marco Tulio Aldana Prillwitz

CONTENIDO

01

INTRODUCCIÓN A HARNESS ENGINEERING

Conocimiento general a IA,
LLM y Context Engineering

02

HISTORIA

Pequeña historia de Harness

03

COMPRENDER

Distinguir harness, contexto,
prompt, RAG y fine-tuning.

04

DISEÑAR

Seleccionar evidencia
relevante para cada paso.

05

EVALUAR

Comparar calidad, latencia y
consumo con un modelo fijo.

06

VERIFICAR

Crear un asistente de
documentación con fuentes y
JSON.

Harness Engineering

- How the whole thing operates

Context Engineering

- What to send the model

Prompt Engineering

- What to ask

INTRODUCCIÓN

El *harness engineering* es el diseño de la infraestructura de software que permite a un modelo de lenguaje de gran tamaño —LLM, por sus siglas en inglés— realizar tareas mediante herramientas, información contextual y mecanismos de verificación. Su propósito es convertir las capacidades del modelo en un proceso de trabajo organizado. Desde la ingeniería en sistemas, este enfoque implica diseñar las condiciones para que un agente pueda interpretar requisitos, ejecutar acciones y comprobar resultados (Lopopolo, 2026).

La relación entre el LLM y el *harness* es complementaria: el modelo interpreta información y propone acciones, mientras que el *harness* administra el ciclo de ejecución y dirige las solicitudes del modelo hacia las herramientas correspondientes. Esta organización permite conectar el razonamiento expresado por el modelo con operaciones concretas, como consultar archivos o ejecutar código (Anthropic, 2026).

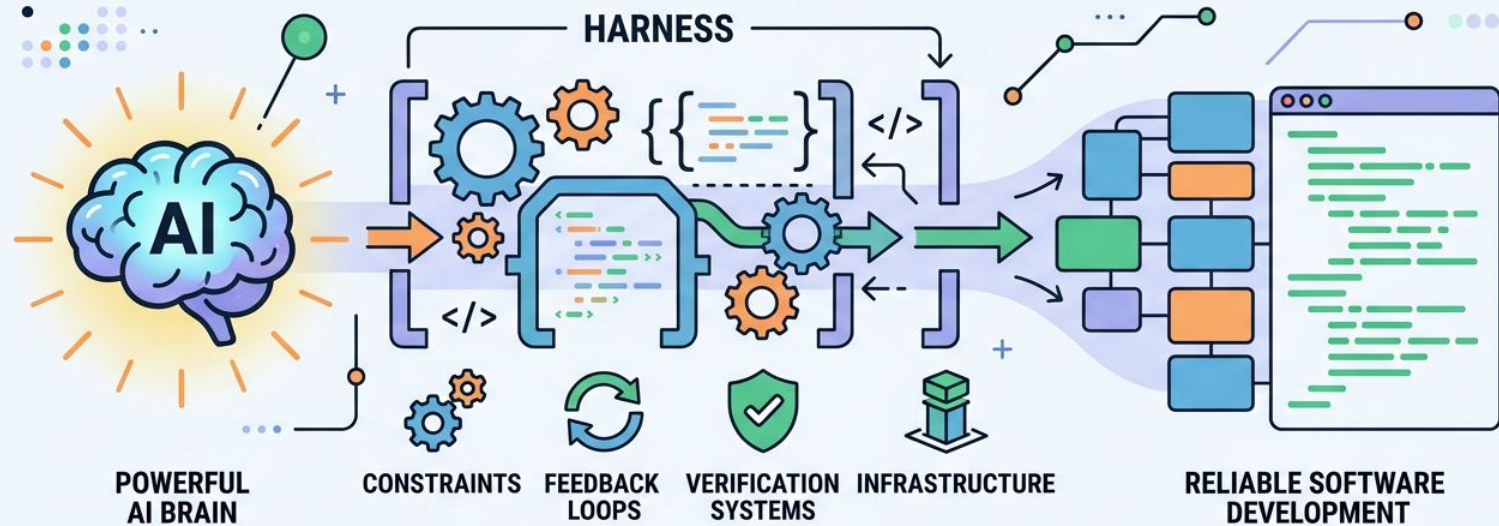
HISTORIA

Un antecedente relevante fue ReAct, presentado en 2022, que propuso alternar razonamiento expresado en texto y acciones para interactuar con fuentes externas. Este trabajo mostró una vía para conectar las respuestas de un LLM con información obtenida durante la ejecución (Yao et al., 2022).

En 2025, Young documentó el uso de *harnesses* para agentes de larga duración, incorporando preparación del entorno, registros de progreso y pruebas. Estas prácticas abordaban dificultades para mantener continuidad entre sesiones y reconocer cuándo una tarea estaba realmente terminada (Young, 2025).

En febrero de 2026, Lopopolo presentó explícitamente el *harness engineering* como un trabajo centrado en diseñar entornos y ciclos de retroalimentación para agentes de programación (Lopopolo, 2026).

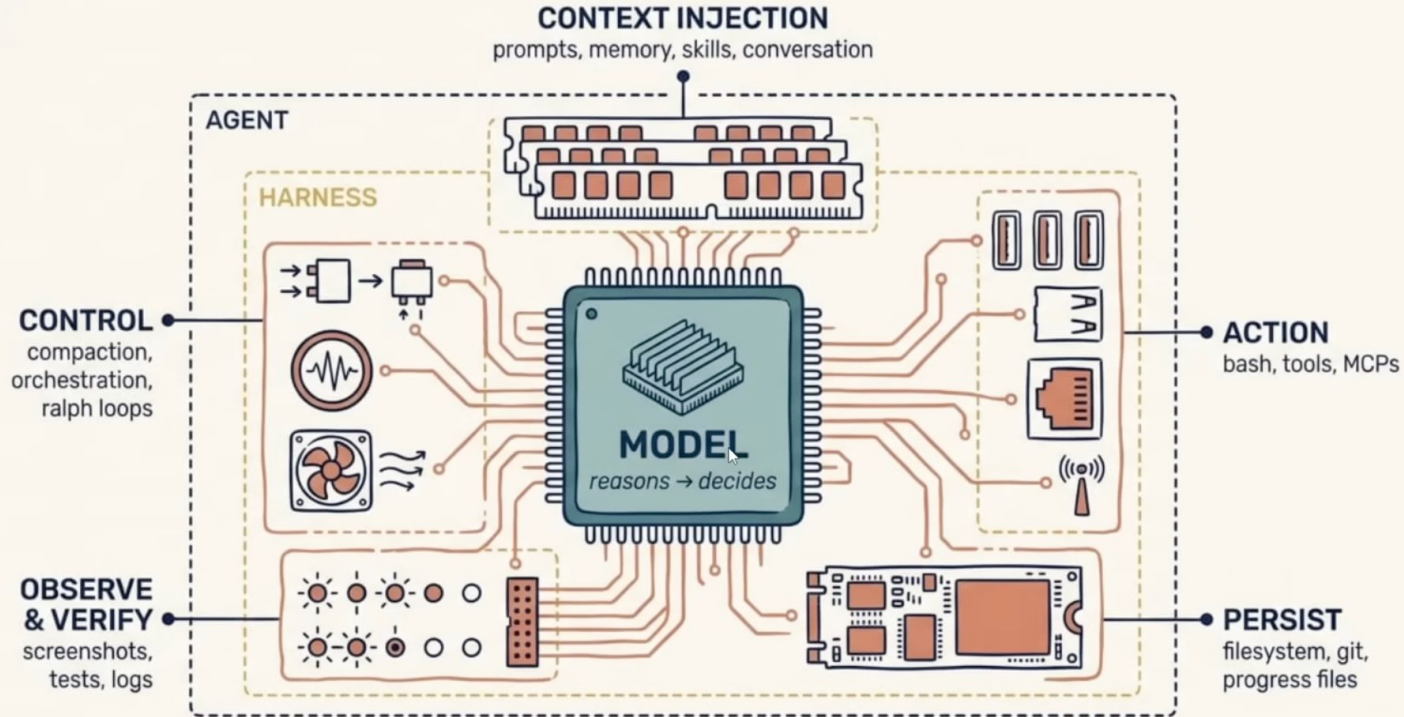
HARNESS ENGINEERING



CONCEPTOS : HARNESS ENGINEERING

El *harness engineering* consiste en diseñar y mantener la infraestructura de software que permite a un modelo de lenguaje de gran tamaño —LLM— participar en tareas que requieren herramientas, información contextual y verificación. Esta infraestructura organiza la ejecución del agente, administra su interacción con el entorno y establece límites operativos. Desde la ingeniería en sistemas, representa el trabajo de convertir las capacidades del modelo en un proceso controlado y evaluable (Lopopolo, 2026).

La relación entre el modelo y el *harness* es complementaria. El LLM interpreta instrucciones y genera respuestas o solicitudes de acciones; el *harness* procesa esas solicitudes, ejecuta las herramientas permitidas y devuelve sus resultados al modelo. Este ciclo permite consultar archivos, modificar código o ejecutar pruebas según las capacidades disponibles. Fazt (2026) explica esta relación mediante la idea de que un agente combina un modelo con la infraestructura que lo rodea.



The model is one chip on the board. The harness is everything else that makes it useful.

VENTAJAS

El *harness engineering* ofrece ventajas para integrar modelos de lenguaje de gran tamaño (LLM) en procesos de desarrollo de software. Al organizar las herramientas, el contexto y los mecanismos de verificación que rodean al modelo, permite convertir sus propuestas en acciones cuyos resultados pueden evaluarse. Desde la ingeniería en sistemas, su principal beneficio es facilitar una ejecución estructurada, con objetivos y criterios de aceptación definidos (Lopopolo, 2026).

Una ventaja es la **retroalimentación durante el trabajo**. El modelo puede recibir los resultados de pruebas y utilizarlos para orientar correcciones. Otra es la **continuidad entre sesiones**: los registros de progreso y el control de versiones ayudan a recuperar el estado del proyecto y reducen la necesidad de reconstruir lo realizado. Young (2025) describe estas prácticas como recursos para abordar tareas extensas mediante avances incrementales.

VENTAJAS

También mejora la **trazabilidad**, porque permite conservar evidencia de las acciones ejecutadas y los resultados obtenidos. Para el ingeniero, esta información facilita revisar cambios y diagnosticar fallos. Asimismo, la integración de herramientas amplía las tareas que el LLM puede realizar dentro de un entorno preparado.

Estas ventajas dependen de la calidad del diseño. Las pruebas insuficientes o el contexto incorrecto pueden conducir a resultados defectuosos. Por ello, considero que el valor del *harness engineering* consiste en combinar automatización con verificación, manteniendo la responsabilidad profesional sobre los requisitos y la calidad del software.

DESVENTAJAS

El *harness engineering* permite organizar herramientas, contexto y controles alrededor de un modelo de lenguaje de gran tamaño (LLM), pero introduce desafíos técnicos. Desde la ingeniería en sistemas, una de sus principales desventajas es la **complejidad adicional**: además de desarrollar la aplicación, es necesario mantener la infraestructura que coordina las acciones del agente y comprobar que sus componentes funcionen correctamente.

Otra limitación es el **aumento del costo y del tiempo de ejecución**. Los procesos que requieren múltiples consultas al modelo, herramientas y ciclos de evaluación pueden consumir más recursos. Schluntz y Zhang (2024) señalan que los sistemas de agentes suelen intercambiar mayor latencia y costo por un mejor desempeño, por lo que su implementación debe justificarse según la tarea.

DESVENTAJAS

También existe el riesgo de **propagar errores**. Una interpretación incorrecta puede influir en acciones posteriores, y las pruebas incompletas pueden llevar a aceptar resultados defectuosos. Además, conservar el contexto entre sesiones sigue siendo un desafío: los registros y resúmenes ayudan, pero requieren una gestión cuidadosa (Young, 2025).

Finalmente, el *harness* necesita adaptarse cuando cambian las capacidades del modelo. Algunas soluciones útiles para una versión pueden volverse innecesarias en otra (Anthropic, 2026). Por ello, considero que su adopción debe equilibrar automatización, mantenimiento y supervisión, evaluando si las mejoras obtenidas compensan la infraestructura y los recursos adicionales.

HERRAMIENTAS DE HARNESS ENGINEERING

Las herramientas de *harness engineering* permiten construir el entorno donde un modelo de lenguaje de gran tamaño (LLM) consulta información, utiliza recursos y recibe retroalimentación. Desde la ingeniería en sistemas, conviene distinguir entre herramientas para construir agentes, protocolos de integración y recursos para verificar resultados.

Claude Agent SDK proporciona componentes para desarrollar agentes, como un ciclo de ejecución, gestión del contexto y herramientas para leer archivos, ejecutar comandos y editar código. El LLM solicita acciones y esta infraestructura facilita su realización (Anthropic, s. f.). Por su parte, **LangGraph** permite organizar procesos con agentes, conservar su estado e incorporar intervención humana durante la ejecución (LangChain, s. f.).

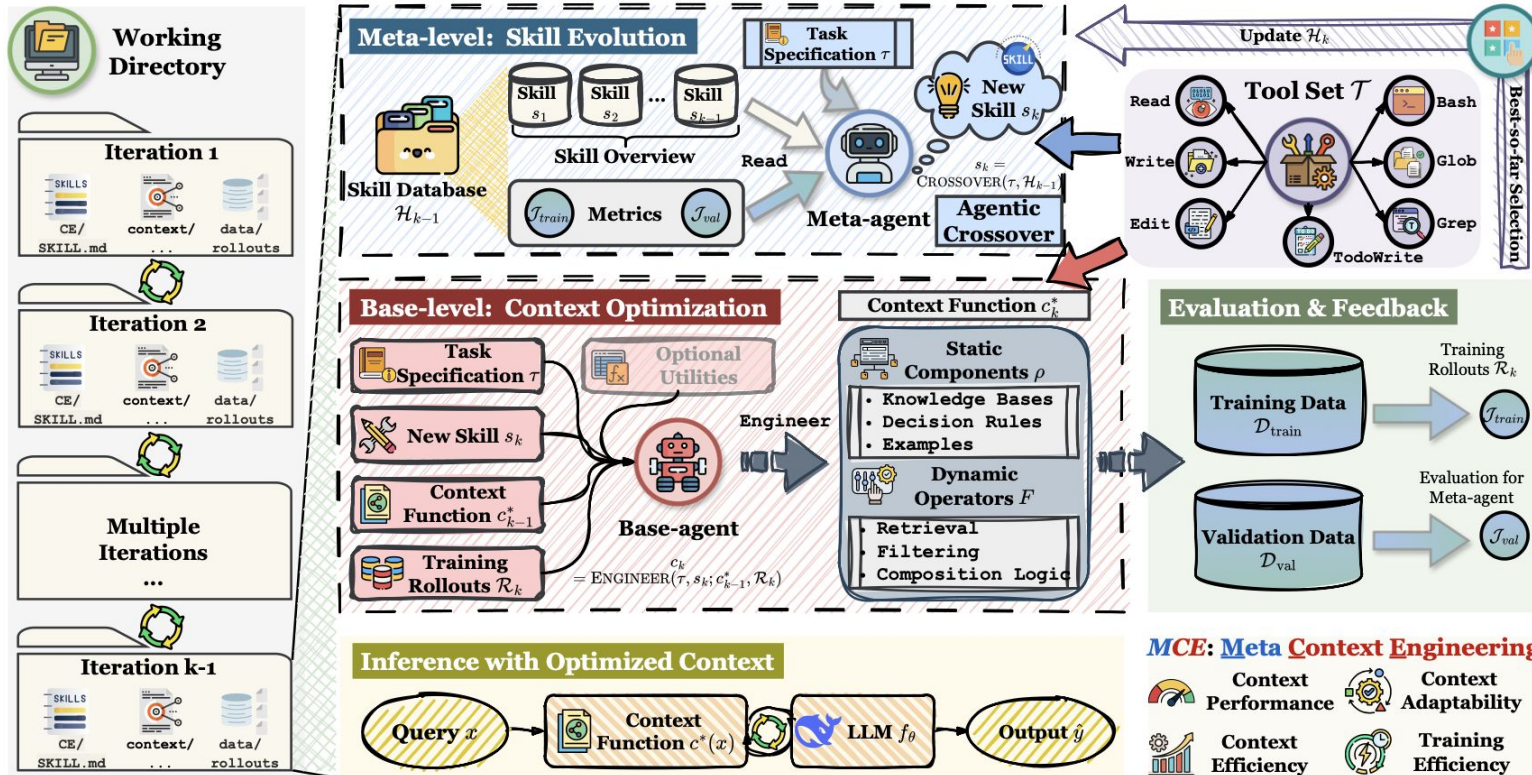
HERRAMIENTAS DE HARNESS ENGINEERING

Otro componente relevante es **Model Context Protocol (MCP)**, un protocolo que estandariza la comunicación con servidores que ofrecen herramientas y recursos. Su función es facilitar integraciones; por sí mismo no constituye un *harness* completo (Model Context Protocol, 2025).

Estas tecnologías se complementan con control de versiones, registros de progreso y pruebas automatizadas. Young (2025) describe cómo estos recursos ayudan a mantener continuidad entre sesiones y comprobar funcionalidades.

Como ingeniero en sistemas, considero que la selección debe responder a las necesidades del proyecto: coordinación, acceso a información, persistencia o validación. El objetivo es construir un entorno donde las acciones del LLM puedan ejecutarse con límites claros y resultados verificables.

CASO DE ÉXITO



CASO DE ÉXITO

Un caso documentado de aplicación exitosa del *harness engineering* es el desarrollo de un producto interno de OpenAI mediante agentes Codex. Lopopolo (2026) informa que el equipo produjo una versión beta utilizada por usuarios reales, con código generado por agentes y dirección humana. El equipo estimó que necesitó aproximadamente una décima parte del tiempo que habría requerido la programación manual; esta cifra corresponde a una estimación interna.

Desde la ingeniería en sistemas, interpreto este caso como un ejemplo de cómo organizar la colaboración entre profesionales y modelos de lenguaje de gran tamaño (LLM). El modelo participa en la interpretación de tareas y generación de soluciones, mientras que el entorno de ejecución proporciona herramientas y evidencia para comprobar los resultados.



CASO DE ÉXITO

La lección práctica es que adoptar agentes requiere definir cómo se reconocerá una solución correcta. En un proyecto propio, establecería requisitos verificables, pruebas pertinentes y registros que permitan revisar los cambios. Estas medidas ayudarían a evaluar si la automatización reduce el trabajo total, incluyendo correcciones y mantenimiento.

El caso ofrece una referencia prometedora, pero su estimación de productividad no debe trasladarse automáticamente a otras organizaciones. Como ingeniero, consideraría exitosa una implementación cuando entregue funcionalidades útiles, mantenga una calidad aceptable y reduzca el esfuerzo necesario para resolver necesidades reales.

BIBLIOGRAFÍA

- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901. <https://arxiv.org/abs/2005.14165>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 4171–4186). Association for Computational Linguistics. <https://doi.org/10.18653/v1/N19-1423>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-T., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. <https://arxiv.org/abs/2005.11401>
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., & Lowe, R. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 27730–27744. <https://arxiv.org/abs/2203.02155>
- Tabassi, E. (2023). Artificial intelligence risk management framework (AI RMF 1.0) (NIST AI 100-1). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.AI.100-1>
- UNESCO. (2021). Recomendación sobre la ética de la inteligencia artificial. <https://www.unesco.org/es/legal-affairs/recommendation-ethics-artificial-intelligence>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30. <https://arxiv.org/abs/1706.03762>



Software Avanzado

Engineering alrededor de LLM

Prompt Engineering · Context Engineering · Harness Engineering · Loop Engineering

LangChain · LangGraph · LangSmith

Objetivos de aprendizaje

- Diferenciar Prompt, Context, Harness y Loop Engineering.
- Diseñar una arquitectura de software que integre un LLM, herramientas, RAG, memoria y validadores.
- Comprender el ciclo Plan → Act → Observe → Evaluate → Correct.
- Relacionar LangChain, LangGraph y LangSmith con sistemas agentic.
- Aplicar estos conceptos a un caso empresarial de análisis de contratos.

Prompt Engineering

Diseño de instrucciones para orientar al modelo

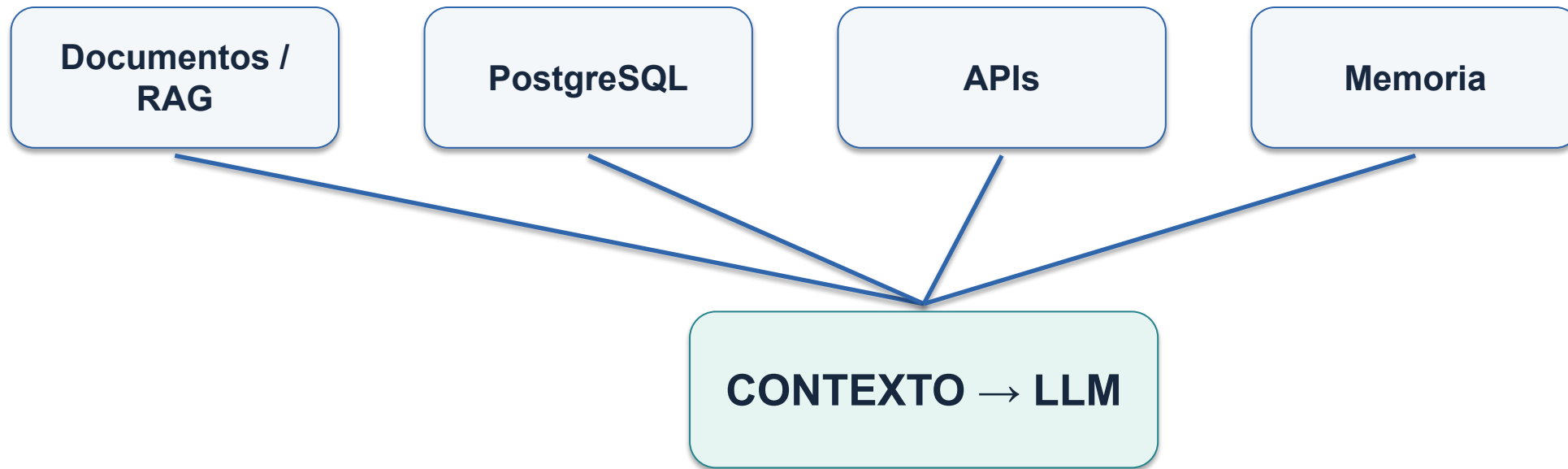
ROL + OBJETIVO + CONTEXTO + RESTRICCIONES + FORMATO + EJEMPLOS

- Define qué se espera del LLM y cómo debe responder.
- Puede especificar audiencia, estructura, longitud, criterios y ejemplos.
- Es necesario, pero no suficiente para construir un sistema confiable.

Pregunta clave: ¿Qué le digo al modelo?

Context Engineering

La información correcta, en el momento correcto



Pregunta clave: ¿Qué necesita saber el modelo para resolver esta tarea?

Harness Engineering

Ingeniería del entorno que rodea al LLM

Prompts

Context

Tools

Memory

Guardrails

Validators

Permissions

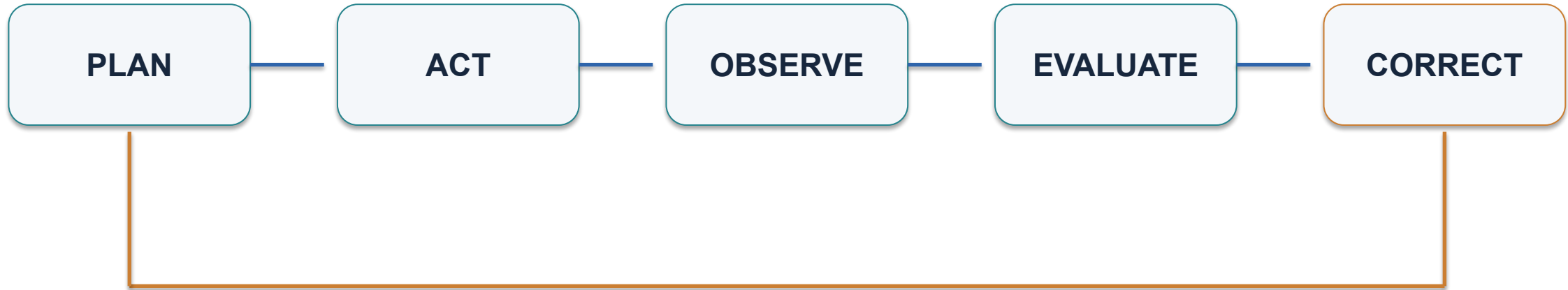
Observability

**LLM dentro de un sistema
controlado**

Pregunta clave: ¿Qué puede hacer el modelo y bajo qué controles?

Loop Engineering

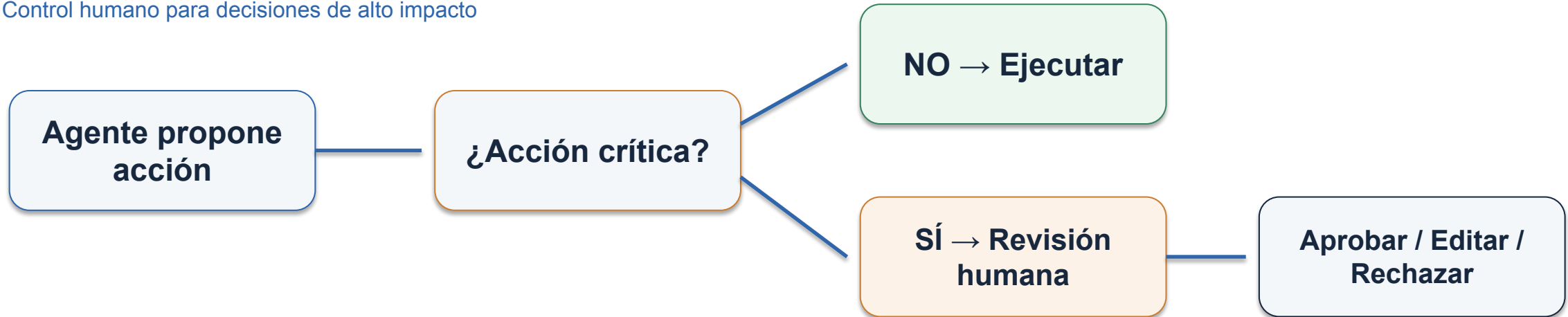
Diseño explícito de ciclos de ejecución y retroalimentación



El loop termina por **SUCCESS**, **ERROR**, **TIMEOUT**, **MAX_ITERATIONS** o aprobación humana.

Human-in-the-Loop

Control humano para decisiones de alto impacto



- Pagos y compras
- Cambios en producción
- Eliminación de datos
- Aprobación contractual

Cuatro capas complementarias

Prompt Engineering

¿Qué le digo?

Prompt

Context Engineering

¿Qué necesita saber?

Contexto

Harness Engineering

¿Qué capacidades y controles tiene?

Sistema

Loop Engineering

¿Cómo verifica y corrige?

Ciclo

Ecosistema LangChain

LangChain

**Models · Prompts · Tools
RAG · Agents · Retrievers**

LangGraph

**State · Nodes · Edges
Loops · Persistence ·
HITL**

LangSmith

**Tracing · Evaluation
Monitoring · Datasets**

Capacidades + Orquestación + Observabilidad

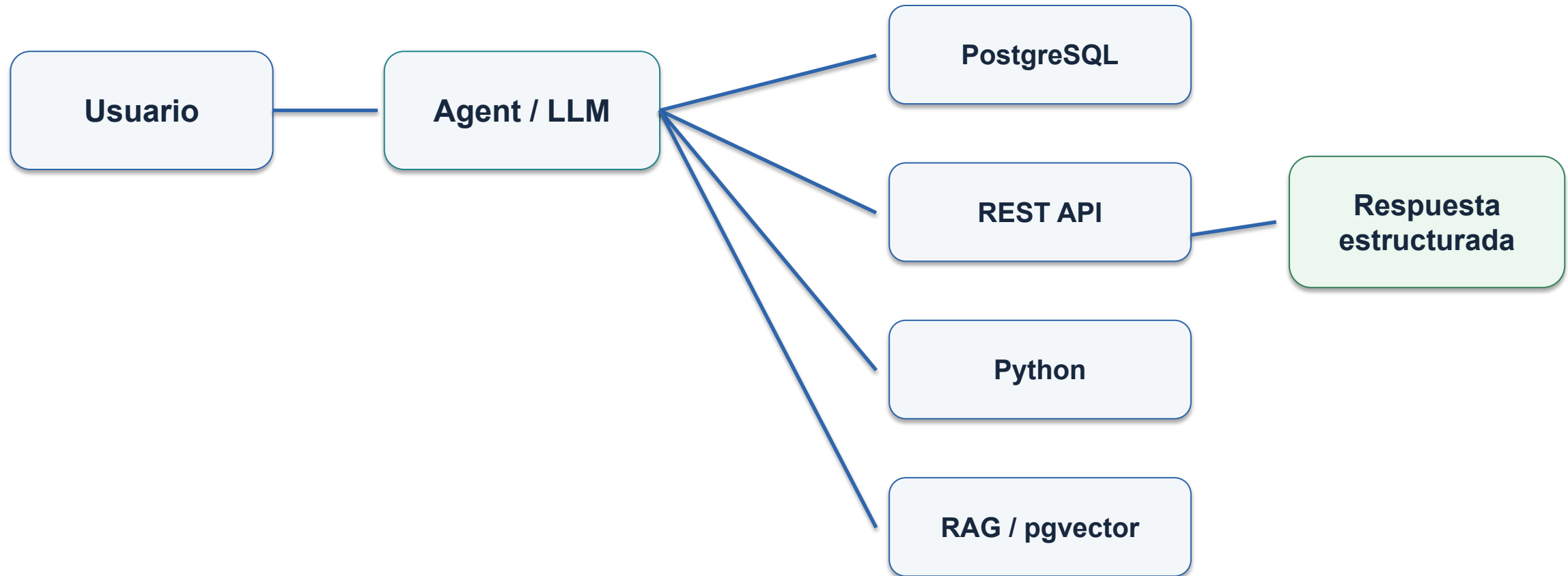
LangChain

Componentes para conectar el LLM con el mundo

- Models: integración con modelos locales o remotos.
- Prompts: construcción de instrucciones.
- Tools: funciones, SQL, APIs, Python y sistemas externos.
- Retrievers / RAG: recuperación de conocimiento.
- Agents: selección dinámica de acciones.
- Structured Output: respuestas con esquemas definidos.

Relación principal: Prompt + Context + Harness Engineering

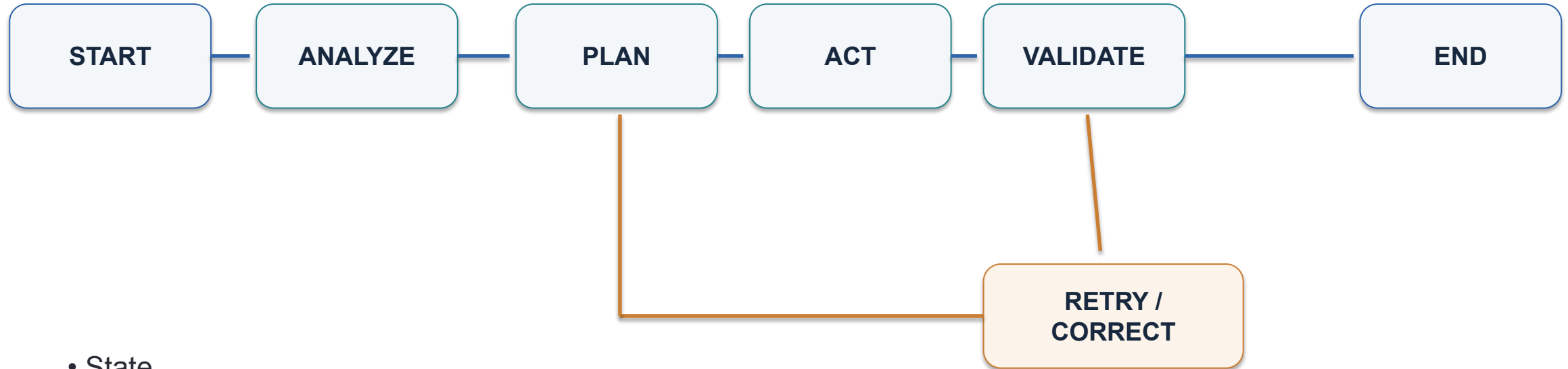
Tools y Agents en LangChain



Una Tool convierte al LLM de generador de texto en componente capaz de consultar y actuar.

LangGraph

Orquestación con estado, decisiones y ciclos



- State
- Nodes
- Edges
- Conditional Edges
- Persistence / Checkpoints
- Human-in-the-Loop

LangSmith

Observabilidad y evaluación del sistema LLM

Tracing

Evaluation

Monitoring

**Datasets /
Experiments**

- Permite observar llamadas al modelo, herramientas, latencia y errores.
- Facilita evaluaciones offline sobre datasets y evaluación de producción.
- Puede combinar reglas determinísticas, revisión humana y LLM-as-a-Judge.
- Convierte el comportamiento del agente en evidencia medible.

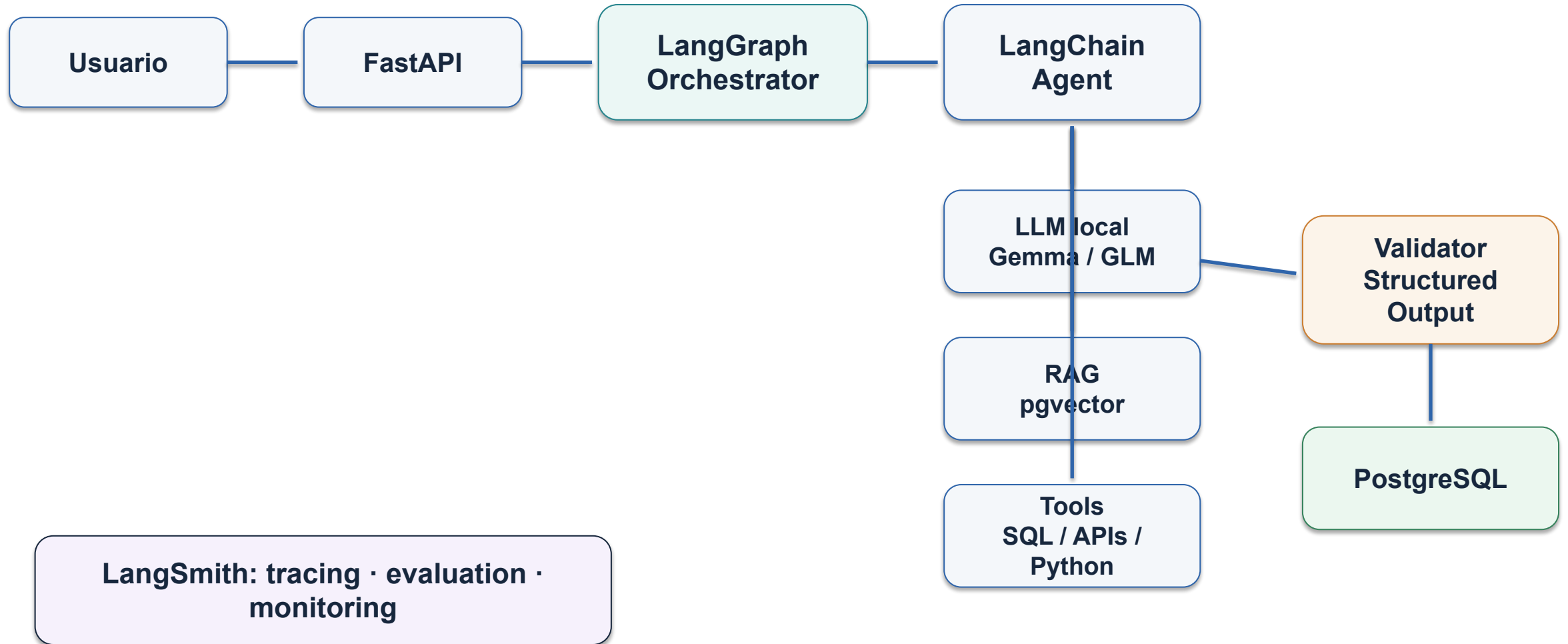
Evaluar un agente: no basta con que “parezca funcionar”



Dataset esperado → Ejecución del agente → Comparación → Métricas → Mejora

- Exactitud de extracción
- Consistencia
- Latencia
- Costo / tokens
- Tasa de reintentos
- Intervenciones humanas

Arquitectura de referencia



Caso práctico: análisis inteligente de contratos

- Extraer proveedor, vigencia, duración y fechas.
- Detectar renovación, garantía, SLA y penalizaciones.
- Identificar entregas parciales y entrega total.
- Generar riesgos y observaciones estructuradas.
- Guardar información y embeddings en PostgreSQL + pgvector.

PDF → Chunking → Embeddings → RAG

LLM → Structured Output → Validator

FAIL → recuperar más contexto → retry

PASS → PostgreSQL → trazabilidad

¿Dónde aparece cada tipo de Engineering?

Prompt Engineering

Instrucciones para extraer campos y producir JSON.

Context Engineering

Cláusulas y fragmentos recuperados del contrato.

Harness Engineering

LLM + RAG + Tools + validadores + permisos.

Loop Engineering

Extraer → validar → recuperar → corregir → revalidar.

Actividad para estudiantes

Escenario: una empresa posee 50,000 contratos PDF y requiere un agente local para analizarlos.

- Diseñar la arquitectura lógica y seleccionar componentes.
- Definir el State y al menos cinco Nodes en LangGraph.
- Definir Tools, permisos y punto de Human-in-the-Loop.
- Establecer MAX_ITERATIONS y criterios SUCCESS / FAILURE.
- Proponer logging, tracing y métricas de evaluación con LangSmith.
- Diseñar al menos 10 casos de prueba para validar la extracción.

Stack tecnológico sugerido para laboratorio

Python

FastAPI

LangChain

LangGraph

LangSmith

**PostgreSQL +
pgvector**

Ollama

Gemma / GLM

Objetivo: que el estudiante construya un sistema reproducible, observable y controlado; no solamente un chatbot.

Síntesis de la clase

PROMPT

¿Qué debe hacer?

CONTEXT

¿Qué debe saber?

HARNESS

¿Qué capacidades y controles tiene?

LOOP

¿Cómo ejecuta, evalúa y corrige?

LangChain + LangGraph + LangSmith → Ingeniería de sistemas agentic

Referencias — APA 7

- Google. (2026). Gemma. Google AI for Developers. <https://ai.google.dev/gemma>
- LangChain. (2026). LangChain overview. <https://docs.langchain.com/oss/python/langchain/overview>
- LangChain. (2026). LangGraph overview. <https://docs.langchain.com/oss/python/langgraph/overview>
- LangChain. (2026). Tools. <https://docs.langchain.com/oss/python/langchain/tools>
- LangChain. (2026). Agents. <https://docs.langchain.com/oss/python/langchain/agents>
- LangChain. (2026). LangSmith observability. <https://docs.langchain.com/langsmith/observability>
- LangChain. (2026). LangSmith evaluation. <https://docs.langchain.com/langsmith/evaluation>
- PostgreSQL Global Development Group. (2026). PostgreSQL documentation. <https://www.postgresql.org/docs/>
- pgvector. (2026). pgvector. <https://github.com/pgvector/pgvector>



LOOP ENGINEERING FINAL

SOFTWARE AVANZADO 2026

Marco Tulio Aldana Prillwitz

CONTENIDO

01

INTRODUCCIÓN A LOOP ENGINEERING

Conocimiento general a IA,
LLM y Context Engineering

02

HISTORIA

Pequeña historia de Harness

03

COMPRENDER

Distinguir harness, contexto,
prompt, RAG y fine-tuning.

04

DISEÑAR

Seleccionar evidencia
relevante para cada paso.

05

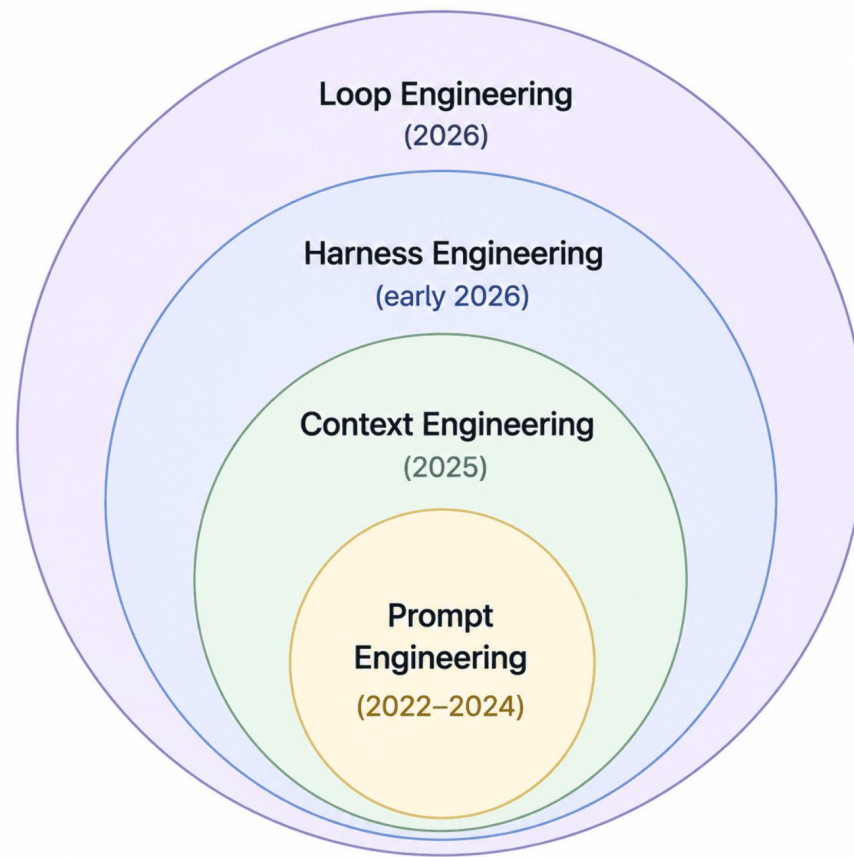
EVALUAR

Comparar calidad, latencia y
consumo con un modelo fijo.

06

VERIFICAR

Crear un asistente de
documentación con fuentes y
JSON.

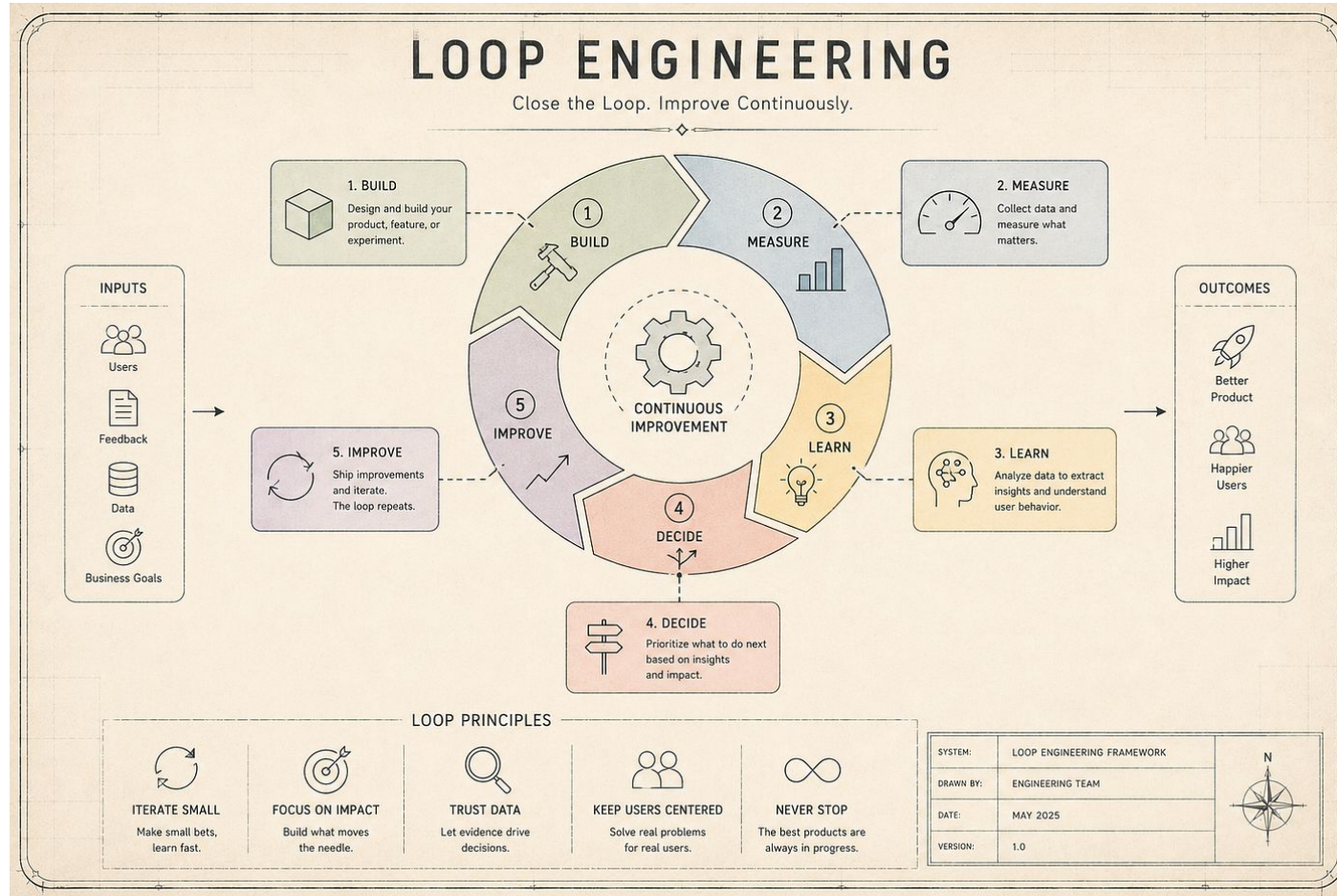


© Each layer contains the one before it.

INTRODUCCIÓN

Loop engineering (ingeniería de bucles) y el context engineering (ingeniería de contextos) son capas complementarias e interconectadas en el diseño de sistemas de agentes de inteligencia artificial autónomos.

Mientras que el context engineering decide qué información ve el modelo en cada turno individual, el loop engineering programa la lógica iterativa de cuántos turnos se ejecutan, cómo se retroalimentan y cuándo se detiene el proceso.



RESUMEN LOOP ENGINEERING

Concepto	Pregunta que responde	Qué diseñamos
Prompt engineering	¿Qué debe hacer el agente en esta tarea?	Instrucciones, restricciones, ejemplos y formato de entrega
Context engineering	¿Qué información necesita para hacerlo bien?	Requisitos, código relevante, contratos, decisiones y resultados previos
Harness engineering	¿En qué entorno puede trabajar y cómo controlamos su ejecución?	Herramientas, permisos, aislamiento, pruebas, registros y persistencia
Loop engineering	¿Cómo avanza, detecta errores y decide cuándo terminar?	

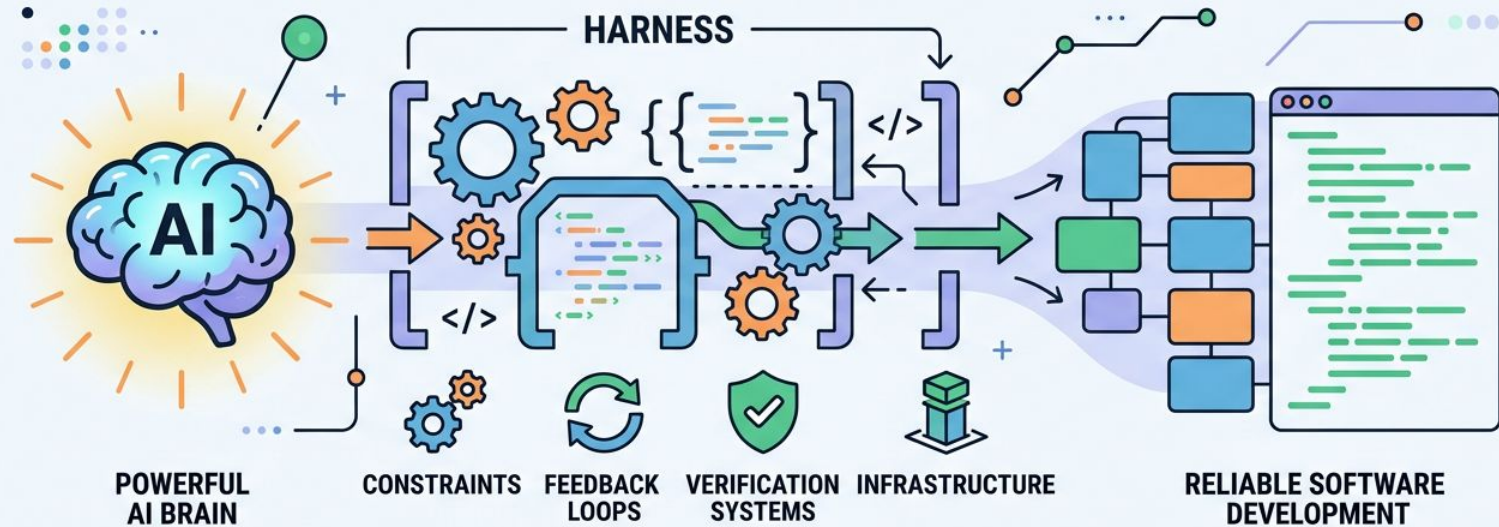
RESUMEN LOOP ENGINEERING

Control de capas: El context engineering opera a nivel micro (el contenido de una sola ejecución o turno), mientras que el loop engineering opera a nivel macro (la secuencia de múltiples ejecuciones orientadas a un objetivo).

El peligro del colapso de memoria: Un bucle largo (loop) mal administrado puede realimentar tantos logs y archivos que sature la ventana de contexto del modelo, provocando alucinaciones.

La solución arquitectónica: Para que un loop funcione de forma autónoma durante horas, el sistema debe usar un buen manejo de contexto (por ejemplo, escribir resúmenes o guardar estados en archivos locales y reglas persistentes) en lugar de acumular todo el historial en el chat volátil.

HARNESS ENGINEERING

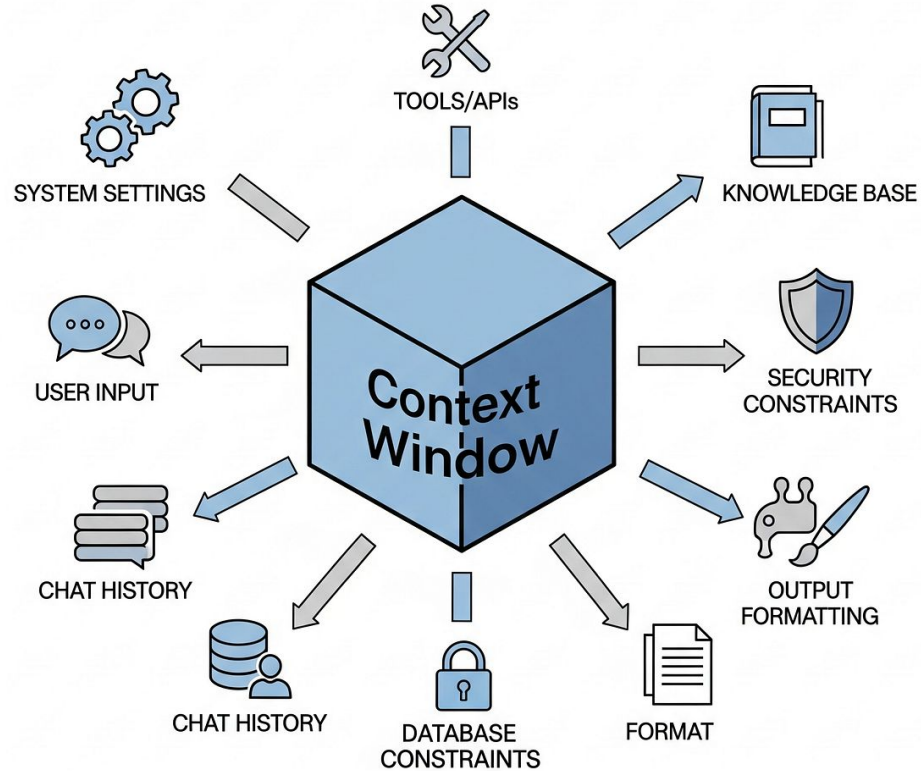


CONCEPTOS : HARNESS ENGINEERING

El *harness engineering* consiste en diseñar y mantener la infraestructura de software que permite a un modelo de lenguaje de gran tamaño —LLM— participar en tareas que requieren herramientas, información contextual y verificación. Esta infraestructura organiza la ejecución del agente, administra su interacción con el entorno y establece límites operativos. Desde la ingeniería en sistemas, representa el trabajo de convertir las capacidades del modelo en un proceso controlado y evaluable (Lopopolo, 2026).

La relación entre el modelo y el *harness* es complementaria. El LLM interpreta instrucciones y genera respuestas o solicitudes de acciones; el *harness* procesa esas solicitudes, ejecuta las herramientas permitidas y devuelve sus resultados al modelo. Este ciclo permite consultar archivos, modificar código o ejecutar pruebas según las capacidades disponibles. Fazt (2026) explica esta relación mediante la idea de que un agente combina un modelo con la infraestructura que lo rodea.

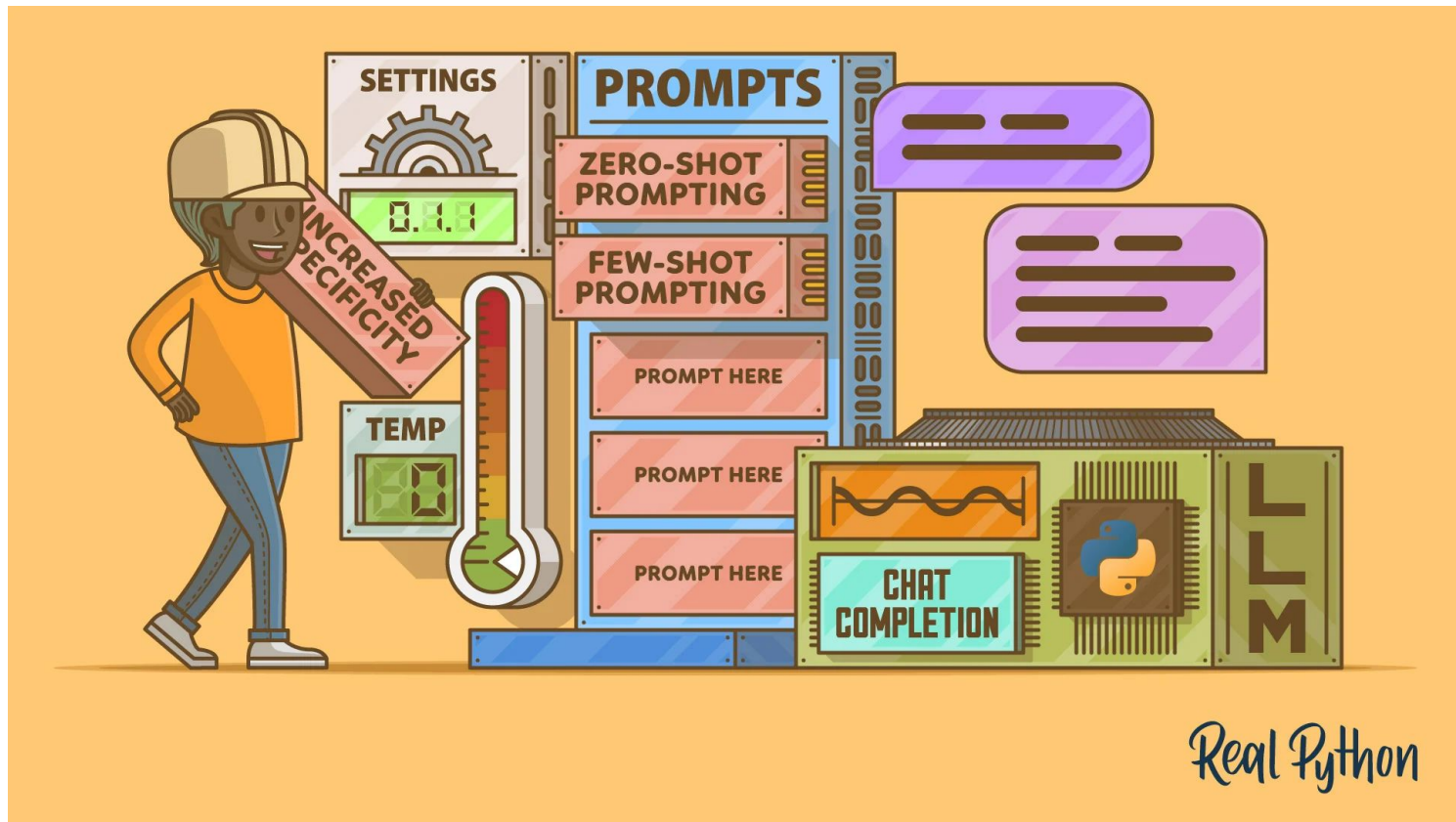
CONTEXT ENGINEERING



CONCEPTOS : HARNESS ENGINEERING

Context Engineering es la disciplina de diseñar, seleccionar y organizar de forma inteligente toda la información que recibe un modelo de inteligencia artificial antes de que genere una respuesta.

A diferencia de la ingeniería de prompts (prompt engineering), que se centra únicamente en escribir una buena pregunta o instrucción estática, la ingeniería de contexto gestiona todo el ecosistema dinámico de datos. Esto incluye el historial de la conversación, documentos adjuntos, instrucciones del sistema y datos recuperados en tiempo real desde bases de conocimiento externas.



CONCEPTOS : PROMPT ENGINEERING

Prompt Engineering es la disciplina de diseñar, estructurar y refinar las instrucciones que se le dan a una inteligencia artificial generativa para obtener respuestas precisas, útiles y relevantes.

Principios :

- Claridad: Escribe instrucciones directas y evita ambigüedades o rodeos innecesarios.
- Contexto: Explica el propósito, los objetivos y la información relevante de la tarea.
- Roles: Asigna un papel específico a la IA para definir su tono y estilo de comunicación.
- Límites: Delimita el formato, la longitud y el alcance del resultado esperado.

Técnicas Comunes : Zero-shot que es preguntar directamente sin ejemplos o Few Shot que es incluir ejemplos previos para guiar al modelo..

- Desglose de tareas: Dividir problemas complejos en subtarefas más pequeñas o usar un enfoque gradual.
- Meta-prompting: Pedirle a la propia inteligencia artificial que ayude a crear o mejorar un prompt.

LLM

LARGE LANGUAGE MODEL



CONCEPTOS : LARGE LOCAL MODELS

Un Modelo de Lenguaje Grande (LLM, por sus siglas en inglés) es un tipo de inteligencia artificial entrenado con enormes cantidades de texto para entender, resumir, traducir y generar lenguaje humano de forma natural.

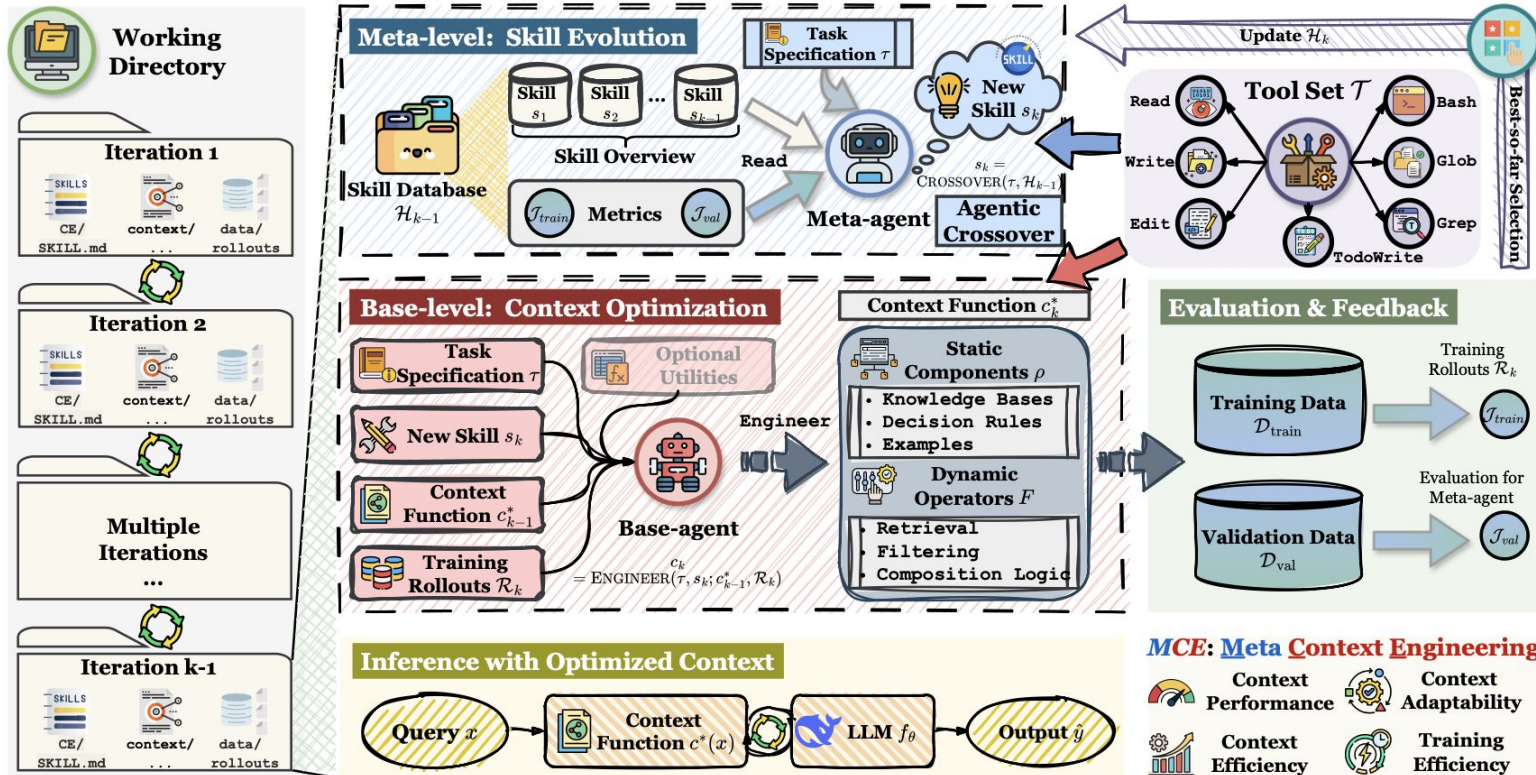
¿Cómo funciona?

Predicción de palabras: El sistema analiza patrones y calcula de manera estadística cuál es la siguiente palabra más probable en una oración.

Redes neuronales: Utiliza una arquitectura avanzada de aprendizaje profundo llamada transformer para capturar el contexto y los matices del idioma.

Entrenamiento masivo: Lee miles de millones de fragmentos de texto (libros, artículos y códigos) para aprender cómo se relacionan las palabras.

CASO DE ÉXITO



CASO DE ÉXITO

Un caso documentado de aplicación exitosa del *harness engineering* es el desarrollo de un producto interno de OpenAI mediante agentes Codex. Lopopolo (2026) informa que el equipo produjo una versión beta utilizada por usuarios reales, con código generado por agentes y dirección humana. El equipo estimó que necesitó aproximadamente una décima parte del tiempo que habría requerido la programación manual; esta cifra corresponde a una estimación interna.

Desde la ingeniería en sistemas, interpreto este caso como un ejemplo de cómo organizar la colaboración entre profesionales y modelos de lenguaje de gran tamaño (LLM). El modelo participa en la interpretación de tareas y generación de soluciones, mientras que el entorno de ejecución proporciona herramientas y evidencia para comprobar los resultados.



CASO DE ÉXITO

La lección práctica es que adoptar agentes requiere definir cómo se reconocerá una solución correcta. En un proyecto propio, establecería requisitos verificables, pruebas pertinentes y registros que permitan revisar los cambios. Estas medidas ayudarían a evaluar si la automatización reduce el trabajo total, incluyendo correcciones y mantenimiento.

El caso ofrece una referencia prometedora, pero su estimación de productividad no debe trasladarse automáticamente a otras organizaciones. Como ingeniero, consideraría exitosa una implementación cuando entregue funcionalidades útiles, mantenga una calidad aceptable y reduzca el esfuerzo necesario para resolver necesidades reales.

BIBLIOGRAFÍA

- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901. <https://arxiv.org/abs/2005.14165>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. En *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 4171–4186). Association for Computational Linguistics. <https://doi.org/10.18653/v1/N19-1423>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-T., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. <https://arxiv.org/abs/2005.11401>
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., & Lowe, R. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 27730–27744. <https://arxiv.org/abs/2203.02155>
- Tabassi, E. (2023). Artificial intelligence risk management framework (AI RMF 1.0) (NIST AI 100-1). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.AI.100-1>
- UNESCO. (2021). Recomendación sobre la ética de la inteligencia artificial. <https://www.unesco.org/es/legal-affairs/recommendation-ethics-artificial-intelligence>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30. <https://arxiv.org/abs/1706.03762>

Mastering LLM Applications with LangChain and Hugging Face

Practical insights into LLM deployment and use cases

Hunaidkhan Pathan
Nayankumar Gajjar



www.bpbonline.com

First Edition 2025

Copyright © BPB Publications, India

ISBN: 978-93-65891-041

All Rights Reserved. No part of this publication may be reproduced, distributed or transmitted in any form or by any means or stored in a database or retrieval system, without the prior written permission of the publisher with the exception to the program listings which may be entered, stored and executed in a computer system, but they can not be reproduced by the means of publication, photocopy, recording, or by any electronic and mechanical means.

LIMITS OF LIABILITY AND DISCLAIMER OF WARRANTY

The information contained in this book is true to correct and the best of author's and publisher's knowledge. The author has made every effort to ensure the accuracy of these publications, but publisher cannot be held responsible for any loss or damage arising from any information in this book.

All trademarks referred to in the book are acknowledged as properties of their respective owners but BPB Publications cannot guarantee the accuracy of this information.

To View Complete
BPB Publications Catalogue
Scan the QR Code:



Dedicated to

*I dedicate this book to my treasured parents, my beloved wife,
my wonderful kids, and my esteemed mentor, **Mr. Amit Saraswat**.*

*Your unwavering support and guidance have been
the cornerstone of my journey.*

– Hunaidkhan Pathan

*Almighty, **Dr. Amit Saraswat**, and My Family*

– Nayankumar Gajjar

About the Authors

- **Hunaidkhan Pathan** currently serves as a Data Science Lead for a leading consulting firm with over a decade of experience in the field. Specializing in machine learning and artificial intelligence, he brings a wealth of expertise to his role. Hunaidkhan holds a PGDM in Data Science from Shanti Business School in Ahmedabad and a degree in Electronics and Communication Engineering from Gujarat Technological University.

He has significantly contributed to the data science community, with his research papers selected and presented at the prestigious SAS Analytics Conference 2013 in Orlando. The titles of his papers include “Marketing Mix Modeling” as an author and “Predicting market uncertainty with Kalman filter” as a co-author. He was also a LinkedIn Top Voice for Data Science and Artificial Intelligence in 2023. He posts regularly on LinkedIn about Generative AI.

Hunaidkhan is an acknowledged **Subject Matter Expert (SME)** in Generative AI and Natural Language Processing. His diverse experience spans various LLM services such as OpenAI, Nvidia Nemo, Anthropic, GCP Generative AI, and AWS Bedrock, in addition to numerous open-source LLMs. His broad experience and profound knowledge make him a valuable contributor in the domain of data science.

- **Nayankumar Gajjar**, has a rich background in Data Science, Machine Learning and Generative AI fields with 9 years of extensive experience as a Data Scientist, Machine Learning Engineer, and Python Developer. Over the years, he has made significant contributions to various high-impact projects, showcasing his expertise in statistical modeling, Generative AI, MLOps, and Cloud Computing. This diverse skill set makes him a versatile and highly skilled professional in the Data Science and Machine Learning domains. He holds a master’s degree in Decision Science, further solidifying his deep understanding of the field. In addition to his professional work, he is a YouTuber and a blogger who shares his experiences and knowledge, offering a complete understanding of statistics and providing detailed coding tutorials. His commitment to education extends to his role as a visiting faculty member, where he has taught Python, SQL, Data Science, and NLP courses. He also co-authored a research paper titled “Thiessen Polygon, A GIS approach for Retail Industry in SAS,” which was presented at the prestigious SAS Analytics Conference 2013 in Orlando.

About the Reviewer

Vijender Singh is a multi-cloud professional with over six years of expertise, currently working in Luxembourg. He holds an MSc with distinction from Liverpool John Moores University, where his research centered on keyphrase extraction. Vijender boasts an impressive collection of cloud certifications, including Google MLPE, five Azure certifications, two AWS certifications, and TensorFlow certification. His role as a technical reviewer for numerous books reflects his commitment to improving the future.

Acknowledgements

We would like to express our sincere gratitude to all those who contributed to the completion of this book.

First and foremost, we extend our heartfelt appreciation to our mentor, Dr. Amit Saraswat, our family and friends for their unwavering support and encouragement throughout this journey. Their love and encouragement have been a constant source of motivation.

We are immensely grateful to BPB Publications for their guidance and expertise in bringing this book to fruition. Their support and assistance were invaluable in navigating the complexities of the publishing process.

We would also like to acknowledge the reviewers, technical experts, and editors who provided valuable feedback and contributed to the refinement of this manuscript. Their insights and suggestions have significantly enhanced the quality of the book.

Last but not least, we want to express our gratitude to the readers who have shown interest in our book. Your support and encouragement have been deeply appreciated.

Thank you to everyone who has played a part in making this book a reality.

Preface

In earlier days, when AI was in its beginning phase, we used to work with static modeling, which contains statistical models like regression, random forest, decision tree, etc. At that time, we used to work with numerical data only, and we did not have much to gain from textual data. Gradually, we got a way under the umbrella of **Bag of Words (BoW)** through which we can work with textual data. The main logic was converting textual data to numerical data. For this, we have a few methods, like count vectors and TF-IDF vectors. These methods create a matrix that shows the occurrence of a word in the given document. Again, these methods were not helping ML models get the context or intent of what had been said in the text. These techniques were helping us to do sentiment analysis and other prediction-based tasks using the above mentioned algorithms.

Fast forward to this time, where we have some advanced techniques like transformers having an underlying architecture of neural networks, due to which ML models are able to get the context as well as the intent of what has been said in the text. This has opened up new opportunities and possibilities in the world of **Natural Language Processing (NLP)** and **Natural Language Generation (NLG)**.

Both NLP and NLG are very important fields in the current era of AI. These fields give machines the power to understand and generate texts like human beings. Some of the readers must have heard the term “ChatGPT,” one of the well-known chatbot platforms from OpenAI. If you have ever used ChatGPT, you must have an idea that it can write code for you, provide medical advice as well, do future prediction as well, and again, here you can chat with ChatGPT, similar to talking to a person and the person answering your questions.

As time passes, these text generation and understanding models become more advanced and able to perform and understand almost all text related tasks. To create such an advancement in the NLP and NLG areas, we will definitely need people who not only know but also have a better understanding of all the terminologies and concepts of NLP and NLG. Also, they should be aware of the steps and phases of the development and deployment of ML models to be served to end users. As we [authors] are interacting with different people in our day-to-day lives, we have found that there is no one step solution that can provide readers with all the above-mentioned things in one place. If readers get terminologies and concepts, then they will not get steps. If they get steps, then there is no practical exposure. If readers have practical exposure, then how to deploy on the cloud is another question. This book comes into the picture in such scenarios.

This book has been written for beginners or people who are stuck at the different stages mentioned in the previous paragraph and do not know about the next steps. This can be divided into three parts. In the first part, you can consider the first three chapters, where we have shown the installation of Python, running Python scripts in different ways, the basic concepts of Python, the installation of editors, and the usage and importance of the virtual environment. In the second part, you can consider chapters 4 and 5, which show the basic and important concepts of NLP and NLG. From chapters 6 to 11, we have shown the usage of important packages like LangChain and Hugging Face. Then we have shown how you can create a chatbot with custom data and integrate it with an application like Telegram. At last, we have shown deployment to an AWS cloud environment. The rest of the chapters are related to future direction and include some useful tips and references.

In this book, we have not only discussed the theoretical approach, but we have also implemented and provided practical exposure as well. In the practical implementation, you will learn all the required steps to be performed to make things work.

We hope that this book will be helpful to any individual who is looking forward to starting their journey in the NLP and NLG fields. We also hope that this book will provide complete guidance and help readers to the required understanding with practical exposure.

Chapter 1: Introduction to Python and Code Editors – In this chapter, readers will learn about Python as a programming language and its history. Readers will get an idea of Python’s features and why it is an important language from an AI/ML perspective. Also, the reader will get an idea about the difference between a code editor and an **Integrated Development Environment (IDE)**.

Chapter 2: Installation of Python, Required Packages, and Code Editors – In this chapter, readers will install Python, all the packages we are going to use throughout the entire book, and an IDE to start with coding. Apart from the installation, readers will gain knowledge on the virtual environment, its importance and its usage. Also, readers will gain knowledge and practical exposure to Python programming basics.

Chapter 3: Ways to Run Python Scripts – In this chapter, readers will create their first Python script, and then they will get practical hands-on experience on different ways to run any Python script.

Chapter 4: Introduction of NLP and its concepts – In this chapter, readers will get exposure to the theoretical concepts and terminologies of NLP, which are essential to start with. Also, readers will get practical hands-on experience with all the important terminologies and concepts.

Chapter 5: Introduction to Large Language Models – This chapter contains theoretical concepts. In this chapter, readers will acquire knowledge on LLM history and its evaluation. Apart from the history, readers will also learn important terminologies and concepts of LLMs.

Chapter 6: Introduction to LangChain, Usage and Importance – In this chapter, readers will gain knowledge of the LangChain package, which is mainly used for text data **Extract, Transform, Load (ETL)** tasks to be later used by LLMs for further processing, understanding, and text generation. Readers will get to know LangChain integration with Hugging Face and how to use LLMs available from Hugging Face. In the chapter, readers will also get practical exposure, which will help them practice and gain confidence.

Chapter 7: Introduction to Hugging Face, its Usage and Importance – In this chapter, readers will get practical exposure to the different LLMs available on Hugging Face Hub and how to use them. Readers will explore Hugging Face Hub as well, which provides a complete ecosystem for LLM deployment.

Chapter 8: Creating Chatbots using Custom Data with Langchain and Hugging Face Hub – In this chapter, readers will create chatbots using the RAG mechanism on custom data using LangChain and Hugging Face combinations. Also, readers will get exposure to the Gradio framework of Hugging Face, through which they can interact with the chatbot created.

Chapter 9: Hyperparameter Tuning and Fine Tuning Pre-Trained Models – In this chapter, the user will gain knowledge about the different hyperparameters available for any LLM, their usage, and how they will impact the LLM's performance.

Chapter 10: Integrating LLMs into Real-World Applications: Case Studies – In this chapter, readers will create a Telegram chatbot with the custom data and interact with it. Readers will get step-by-step guide on the implementation.

Chapter 11: Deploying LLMs in Cloud Environments for Scalability – In this chapter, readers will get a step-by-step guide to deploying chatbots and LLM models in an AWS cloud environment. Readers will also get an idea about GCP.

Chapter 12: Future Directions: Advances in LLMs and Beyond – In this chapter, readers will learn future directions and where to go from here once the book has been completed.

Appendix A: Useful Tips for Efficient LLM Experimentation – In this chapter, we have shared some tips to use LLMs more efficiently.

Appendix B: Resources and References – In this chapter, we have provided some of the resources and references for the readers to get more depth and detailed knowledge on different models and packages.

Code Bundle and Coloured Images

Please follow the link to download the
Code Bundle and the *Coloured Images* of the book:

<https://rebrand.ly/bf9408>

The code bundle for the book is also hosted on GitHub at
<https://github.com/bpbpublications/Mastering-LLM-Applications-with-LangChain-and-Hugging-Face>.
In case there's an update to the code, it will be updated on the existing GitHub repository.
We have code bundles from our rich catalogue of books and videos available at
<https://github.com/bpbpublications>. Check them out!

Errata

We take immense pride in our work at BPB Publications and follow best practices to ensure the accuracy of our content to provide with an indulging reading experience to our subscribers. Our readers are our mirrors, and we use their inputs to reflect and improve upon human errors, if any, that may have occurred during the publishing processes involved. To let us maintain the quality and help us reach out to any readers who might be having difficulties due to any unforeseen errors, please write to us at :

errata@bpbonline.com

Your support, suggestions and feedbacks are highly appreciated by the BPB Publications' Family.

Did you know that BPB offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at www.bpbonline.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at :

business@bpbonline.com for more details.

At **www.bpbonline.com**, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on BPB books and eBooks.

Piracy

If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at **business@bpbonline.com** with a link to the material.

If you are interested in becoming an author

If there is a topic that you have expertise in, and you are interested in either writing or contributing to a book, please visit **www.bpbonline.com**. We have worked with thousands of developers and tech professionals, just like you, to help them share their insights with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Reviews

Please leave a review. Once you have read and used this book, why not leave a review on the site that you purchased it from? Potential readers can then see and use your unbiased opinion to make purchase decisions. We at BPB can understand what you think about our products, and our authors can see your feedback on their book. Thank you!

For more information about BPB, please visit **www.bpbonline.com**.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



Table of Contents

1. Introduction to Python and Code Editors.....	1
Introduction	1
Structure	1
Objectives	1
Introduction to Python.....	2
Introduction to code editors	5
Conclusion	6
References.....	7
Further reading.....	7
2. Installation of Python, Required Packages, and Code Editors.....	9
Introduction	9
Structure	9
Objectives	10
General instructions.....	10
Installation of Python on Windows.....	11
Installation of Python on Linux	13
Installation of Python on MacOS.....	13
<i>Using Docker for Python</i>	<i>14</i>
<i>Installation of IDE.....</i>	<i>15</i>
Installation of PyCharm	16
Installation of required packages.....	17
<i>Virtual environment.....</i>	<i>17</i>
<i>virtualenv.....</i>	<i>18</i>
<i>pipenv.....</i>	<i>19</i>
<i>Folder structure.....</i>	<i>19</i>
<i>Creating a virtual environment</i>	<i>21</i>
<i>PEP 8 standards.....</i>	<i>23</i>

Following PEP 8 in PyCharm	25
Object-Oriented Programming concepts in Python.....	26
Classes in Python	29
Functions in Python.....	32
For loop in Python	33
While loop in Python	34
If-else in Python	35
Conclusion	36
3. Ways to Run Python Scripts	37
Introduction	37
Structure	37
Objectives	38
Setting up the project.....	38
Running Python scripts from PyCharm	42
Running Python Scripts from Terminal	47
Running Python scripts from Jupyter Lab and Notebook.....	49
Running Python Scripts from Docker	52
Conclusion	54
4. Introduction to NLP and its Concepts.....	55
Introduction	55
Structure	55
Objectives	56
Natural Language Processing overview.....	56
Key concepts	56
Corpus	57
N-grams.....	59
Tokenization.....	63
Difference in tokens and n-grams	67
Stop words removal	67
Stemming	70

<i>Lemmatization</i>	70
<i>Lowercasing</i>	74
<i>Part-of-speech tagging</i>	74
<i>Named Entity Recognition</i>	76
<i>Bag of words</i>	79
<i>Word embeddings</i>	83
<i>Topic modeling</i>	86
<i>Sentiment analysis</i>	88
Large language models	90
<i>Transfer learning</i>	91
Text classification	91
Prompt engineering	96
<i>Hallucination</i>	96
<i>Syntactic relationship</i>	96
<i>Semantic relationship</i>	96
Conclusion	97
5. Introduction to Large Language Models	99
Introduction	99
Structure	99
Objectives	100
History	100
LLM use cases	102
LLM terminologies	104
Neural networks	111
Transformers	116
Pre-built transformers	127
<i>Bidirectional Encoder Representations from Transformers</i>	128
<i>Generative Pre-trained Transformer</i>	129
<i>Text-to-text transfer transformer</i>	130
<i>DistilBERT</i>	130
<i>XLNet</i>	131

<i>RoBERTa</i>	132
Conclusion	132
Further readings.....	133
References.....	134
6. Introduction of LangChain, Usage and Importance	135
Introduction	135
Structure	135
Objectives	136
LangChain overview	136
Installation and setup	137
Usages.....	140
Opensource LLM models usage	140
Data loaders	151
Opensource text embedding models usage	153
Vector stores	162
Model comparison	169
Evaluation	175
<i>Types of evaluation</i>	175
Conclusion	194
Points to remember	195
References.....	195
7. Introduction of Hugging Face, its Usage and Importance	197
Introduction	197
Structure	197
Objectives	198
Exploring the Hugging Face platform	198
Installation and setup	203
Datasets.....	203
Usage of opensource LLMs	213
Generating vector embeddings.....	222
Evaluation	225

Transfer learning with Hugging Face API.....	232
Real-world use cases of Hugging Face	234
Conclusion	234
References.....	235
8. Creating Chatbots Using Custom Data with LangChain and Hugging Face Hub	237
Introduction	237
Structure	238
Objectives	238
Setup	238
Overview	238
Steps to create RAG based chatbot with custom data	242
Dolly-V2-3B details	255
Data loaders by LangChain	255
Vector stores by LangChain.....	256
Conclusion	256
References.....	257
9. Hyperparameter Tuning and Fine Tuning Pre-Trained Models.....	259
Introduction	259
Structure	260
Objectives	260
Hyperparameters of an LLM.....	260
Hyperparameters at inferencing or at text generation	263
Fine-tuning of an LLM	267
Data preparation for finetuning an LLM.....	267
Performance improvement.....	275
Conclusion	275
References.....	276
10. Integrating LLMs into Real-World Applications – Case Studies	277
Introduction	277
Structure	277

Objectives	278
Case studies	278
Use case with Telegram.....	280
<i>Setup</i>	281
Conclusion	289
References.....	290
11. Deploying LLMs in Cloud Environments for Scalability	291
Introduction	291
Structure	292
Objectives	292
Amazon Web Services	292
<i>Step 1: Creating an Amazon SageMaker Notebook Instance</i>	293
<i>Step 2: Create folders in SageMaker to store data</i>	297
<i>Step 3: Create vector embeddings</i>	299
<i>Step 4: Auto scaling</i>	309
Google Cloud Platform	311
Conclusion	311
References.....	312
12. Future Directions: Advances in LLMs and Beyond.....	313
Introduction	313
Structure	313
Objectives	314
Generative AI market growth	314
Reasoning	314
Emergence of multimodal models.....	315
Small domain-specific models.....	315
<i>Multi agent framework</i>	317
Quantization and Parameter-Efficient Fine Tuning	317
Vector databases	318
Guardrails.....	319
Model evaluation frameworks.....	320

Ethical and bias mitigation	324
Safety and security	325
Conclusion	326
References	327
Appendix A: Useful Tips for Efficient LLM Experimentation	329
Structure	329
Objectives	330
Understanding the challenges of LLM experimentation	330
Preparing data for LLM experimentation	332
Optimizing model architecture and hyperparameters	333
Efficient training strategies for LLMs	335
Evaluating and interpreting experimental results	336
Fine-tuning for specific applications	338
Scaling up: Distributed training and parallel processing	339
Deployment considerations for LLMs	341
Conclusion	343
References	343
Appendix B: Resources and References	345
Introduction	345
Books and articles	345
Research papers	346
LangChain resources	347
Hugging Face resources	347
<i>Alternative resources to LangChain</i>	348
Community and support	350
Other important resources	350
Conclusion	350
Index	351-356

CHAPTER 1

Introduction to Python and Code Editors

Introduction

Python is a really powerful programming language that is simple and easy to read. This language is being used in many technology areas. There are rules called **Python Enhancement Proposal (PEP)** standards, which help you write proper Python code. These rules give instructions for how we should write and develop the programming language which helps us keep our code clean and of high quality!

There are also different ways that we can work on our Python codes - either through code editors or **Integrated Development Environments (IDEs)**.

Structure

In this chapter we will discuss the following topics:

- Introduction to Python
- Introduction to code editors

Objectives

Learning Python well is a great start for getting into generative AI. Python is known for being easy to understand and has lots of tools you can use. It is a good language to learn

if you want to understand how programming works. Since Python is used everywhere, it is important for learning different kinds of machine learning, which is really helpful if you are interested in generative AI. If you are good at Python, you can easily use the important tools and information you need for generative AI. This makes it easier to move on to more advanced things like understanding how computers understand human language or deep learning.

Introduction to Python

Created by *Guido Van Rossum* back in the late 1980s, today, almost everybody uses Python! Here is a brief introduction to Python:

- **Readability:** It is easy to understand any code written by another person due to its simplicity.
- **Interpreted language:** Python is an interpreted language. You do not need to compile your program before running it on the system. This makes development faster and easier, as you can execute code line by line, and easy to debug.
- **Cross-platform:** Your computer runs Windows? MacOS? Linux? Do not worry! No matter what operating system your computer has installed, they all support Python!
- **Versatility:** Many more useful features like versatility, object-oriented libraries, and interfaces, community support, dynamic variable declaration, etc., which make work super smooth.
- **Libraries and frameworks:** If asked to paint an image without a canvas color brush, it is hard for anyone. In similar cases with computer programming, we require a lot of tools to make it happen; IDEs and code editors are one of them, which have unique functionality based on requirements and can be customized accordingly. For example, PyCharm is a Python-specific editor. We also have something called Jupyter Notebook or Jupyter Lab in the Python world. Python provides a large collection of different libraries for different tasks. For example, Django is used for web development, NumPy and Pandas are used for data analysis, and Tensorflow and Keras are used for deep learning.
- **Community and support:** Python's simplicity and community support make it a highly usable coding language. It does not matter if you are new to programming or an experienced pro: Python has something for everyone! This powerful tool will always come in handy whether it is web development, data analysis or AI/ML tasks etc.
- **Open source:** Python is open-source and free to use. This encourages collaboration and innovation, as anyone can contribute to the language's development or create their own Python packages.
- **Object-oriented:** Python is an object-oriented programming language, which means code organization around objects/classes simplifies managing complex systems.

- **Dynamic typing:** Python uses dynamic typing, which means variables do not require any explicit type definition speeding up development but further necessitates attention towards avoiding potential typing errors.
- **High-level language:** Python is a high-level programming language; since lower-level complexities are abstracted away, users can concentrate mainly on problem-solving and worrying less about underlying hardware details.
- **Duck typing:** Python follows the principle of *duck typing*, which means the object type determination is based on its behavior which gives code the freedom to be concise yet vigilant towards object compatibility.
- **Multi-paradigm:** Python supports multiple programming instances, including procedural, object-oriented, and functional programming. This versatility allows us to choose the most suitable approach for our project's requirements.
- **Interoperability:** Python interacts smoothly with other languages like C, C++, Java, etc., enabling utilization of existing libraries/code.
- **Popular use cases:** Usable across several domains, i.e., web development (using Django/Flask/FastAPI), data science (machine learning using Scikit-Learn/TensorFlow), Scientific computing (using NumPy/SciPy), automation scripting or even game development, the list is endless.
- **Python 2 vs. Python 3:** It is important to note that there are two major versions of Python: Python 2 and Python 3. Though there are mainly two versions available, as of Jan 1st, 2020, only Python 3 receives updates/supports - have an edge by starting all new projects in this version!

In conclusion, regardless of whether you are a beginner embarking upon an initial language learning journey or an experienced developer handling intricate setup, the simplicity/readability/use case versatility/community backup is positioning Python as handy across several coding tasks.

Zen (**Python Enhancement Proposal PEP 20**) Philosophy embraces design ideals/principles defining how Python code should be written for not just computers but easy understanding by fellow developers too!

Here are some of the key principles from the *Zen of Python* written by *Tim Peters*:

- **Beautiful is better than ugly:** Python code should be aesthetically pleasing, clear, and elegant. This encourages developers to write code that is not only functional but also visually appealing.
- **Explicit is better than implicit:** Code should be explicit in its intentions and behavior. Avoid relying on hidden or implicit features to make the code more understandable.
- **Simple is better than complex:** Simplicity is preferred over complexity. The code should be straightforward and easy to understand rather than unnecessarily convoluted.

- **Complex is better than complicated:** While simplicity is encouraged and complexity is necessary, it should be well-structured and not overly complicated. Complex code should have a clear purpose and design.
- **Flat is better than nested:** Deeply nested code structures should be avoided. Keeping code relatively flat, with fewer levels of indentation, makes it more readable and maintainable.
- **Sparse is better than dense:** Code should be spaced out and not overly dense. Proper spacing and indentation enhance readability.
- **Readability counts:** Readability is a top priority in Python. Code should be written with the goal of making it easy to read and understand, not just for the computer but also for other developers.
- **Special cases are insufficient to break the rules:** Consistency is important. While there may be exceptional cases, they should not lead to a violation of established coding conventions and rules.
- **Practicality beats purity:** While adhering to best practices and principles is important, practicality should not be sacrificed in the pursuit of theoretical perfection. Real-world solutions sometimes require pragmatic compromises.
- **Errors should never pass silently. Unless explicitly silenced:** Errors and exceptions should be handled explicitly. If you encounter an error, it should not be ignored or suppressed unless you have a good reason to do so.
- **In the face of ambiguity, refuse the temptation to guess:** When faced with uncertainty or ambiguity in your code, it is better to be explicit and not make assumptions. Clarity should prevail.
- **There should be one and preferably only one obvious way to do it:** Python encourages a single, clear way to accomplish tasks to minimize confusion and inconsistency in code.
- **Although that way may not be obvious at first unless you are Dutch:** This light-hearted remark acknowledges that not all design decisions may immediately make sense to everyone and hints at Python's creator, *Guido van Rossum*.
- **Now is better than never. Although never is often better than right now:** While taking action is important, rushing without proper consideration can lead to errors. It's a reminder to balance speed with careful thought.
- **If the implementation is hard to explain, it is a bad idea. If the implementation is easy to explain, it may be a good idea:** Code should be designed in a way that makes its purpose and behavior clear and straightforward. Complex, hard-to-explain implementations should be avoided.
- **Namespaces are one honking great idea: Let us do more of those:** Encouragement to use namespaces for organizing and managing variables and functions, promoting modularity, and avoiding naming conflicts.